

Database

A database is an organized collection of data that is stored and accessed electronically. It allows for efficient data management, retrieval, and manipulation using systems like SQL-based relational databases or NoSQL alternatives.

Types of Databases

Relational DB Data stored in structured format of rows and column (table) and language used to interact with is Structured Query Language.

Examples: MySQL , PostgreSQL , Oracle

NoSQL DB Data stored in unstructured format like .json , graph , key:value pair etc.

Examples : MongoDB , Firebase , Cassandra

SQL Commands & Queries

SQL Commands are instructions used to perform specific tasks in a database, such as creating tables, inserting data, or modifying structures.

They are grouped into categories like DDL, DML, DCL, and TCL.

SQL Queries are specific types of SQL commands used to retrieve data from a database, typically using the SELECT statement with conditions and filters.

SQL Commands basic types and functions:

Task	Command
Permission Management	GRANT , USE db_name
Creating Tables	CREATE TABLE
Modifying Tables	ALTER TABLE
Data Manipulation	INSERT INTO , UPDATE , DELETE

Some basic syntaxes:

Syntax	Description
CREATE DATABASE db_name;	creates a new DB
DROP DATABASE db_name;	deletes the created DB
USE db_name;	to select the desired DB to work with

Schema

A schema is a structured framework or blueprint that defines how data is organized and how relationships between data are handled. In databases, it specifies tables, fields, data types, and constraints.

Example:

```
CREATE TABLE Students (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

This schema defines the structure of the Students table with columns and data types.

Primary Key

A Primary Key is a unique identifier for each record in a database table; it ensures that no two rows have the same value in that column and does not allow null.

Example:

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

StudentID uniquely identifies each student.

Foreign Key

A Foreign Key is a field in one table that links to the Primary Key in another table, establishing a relationship between the two tables.

Example:

```
CREATE TABLE Enrollments (  
    EnrollmentID INT,  
    StudentID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
);
```

StudentID in Enrollments refers to StudentID in Students, linking the two tables.

Normalization

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves dividing large tables into smaller, related tables and defining relationships between them.

Data Types

Data Types in MySQL define the kind of data that can be stored in a column. They specify whether a column holds numbers, text, dates, etc., helping the database manage and validate the data correctly.

Example

```
CREATE TABLE Employees (  
  ID INT,  
  Name VARCHAR(50),  
  HireDate DATE,  
  Salary DECIMAL(10,2)  
);
```

INT → for whole numbers

VARCHAR(50) → for text up to 50 characters

DATE → for date values

DECIMAL(10,2) → for numbers with decimals (e.g., salary)

Data Type in SQL	Range / Size Limit	Definition	When to Use	Example
<code>CHAR(n)</code>	0 to 255 characters	Fixed-length string	Use when all values are same length	<code>CHAR(10)</code> for fixed ID codes
<code>VARCHAR(n)</code>	0 to 65,535 bytes*	Variable-length string	Use when string length varies	<code>VARCHAR(50)</code> for names
<code>TINYTEXT</code>	Up to 255 bytes	Small text string	For short notes or messages	Short comment field
<code>TEXT</code>	Up to 65,535 bytes	Medium-length text	For paragraphs or article content	Product descriptions
<code>MEDIUMTEXT</code>	Up to 16,777,215 bytes	Longer text string	For large text data	Blog posts or documentation
<code>LONGTEXT</code>	Up to 4,294,967,295 bytes	Very large text	For large documents or logs	Books, logs
<code>TINYBLOB</code>	Up to 255 bytes	Small binary data	Store small files or images	Small icons
<code>BLOB</code>	Up to 65,535 bytes	Binary Large Object	Store medium-sized binary files	Uploaded images
<code>MEDIUMBLOB</code>	Up to 16MB	Larger binary data	Audio/video files	Audio clips
<code>LOBLOB</code>	Up to 4GB	Very large binary	Store large multimedia files	Movies, backups

Data Type in SQL	Range (Signed)	Definition	When to Use	Example
<code>TINYINT</code>	-128 to 127	Very small integer (1 byte)	When storing small numbers (e.g., flags, age, status)	<code>TINYINT</code> for <code>isActive</code> (0/1)
<code>SMALLINT</code>	-32,768 to 32,767	Small integer (2 bytes)	For small ranges like quantity or short IDs	<code>SMALLINT</code> for number of items
<code>MEDIUMINT</code>	-8,388,608 to 8,388,607	Medium integer (3 bytes)	When <code>SMALLINT</code> is too small but <code>INT</code> is too big	<code>MEDIUMINT</code> for user IDs
<code>INT</code> or <code>INTEGER</code>	-2,147,483,648 to 2,147,483,647	Standard integer (4 bytes)	Commonly used for general-purpose counting or IDs	<code>INT</code> for employee ID
<code>BIGINT</code>	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Large integer (8 bytes)	For very large numbers like large user counts or money in cents	<code>BIGINT</code> for bank balance in cents

✓ Notes:

Use UNSIGNED to double the positive range (e.g., TINYINT UNSIGNED: 0 to 255).

Choose the smallest type that fits your data to save space and improve performance.

Decimal / Float Data Types in MySQL

Data Type in SQL	Range / Precision	Definition	When to Use	Example
<code>DECIMAL(p, s)</code> or <code>NUMERIC(p, s)</code>	Exact values, up to 65 digits	Fixed-point number with precision (<code>p</code>) and scale (<code>s</code>)	For exact values like money or rates	<code>DECIMAL(10,2)</code> for currency (e.g., 99999999.99)
<code>FLOAT(p)</code>	~7 digits precision	Approximate, single-precision floating point	When storage is more important than exact precision	<code>FLOAT</code> for temperature or scientific data
<code>DOUBLE</code> or <code>DOUBLE PRECISION</code>	~15 digits precision	Approximate, double-precision floating point	For more accurate scientific or large-range data	<code>DOUBLE</code> for coordinates or measurements
<code>REAL</code>	Synonym for <code>DOUBLE</code> (in MySQL)	Same as <code>DOUBLE</code>	Same as above	<code>REAL</code> for latitude/longitude

Date / Time Data Types in MySQL

Data Type in SQL	Range	Definition	When to Use	Example
<code>DATE</code>	'1000-01-01' to '9999-12-31'	Stores only date (YYYY-MM-DD)	For birthdays, due dates, etc.	<code>DATE</code> for <code>birth_date</code>
<code>DATETIME</code>	'1000-01-01 00:00:00' to '9999-12-31 23:59:59'	Date and time (no timezone)	For events with both date and time	<code>DATETIME</code> for order time
<code>TIMESTAMP</code>	'1970-01-01 00:00:01' UTC to '2038-01-19 03:14:07' UTC	Date and time in UTC, auto-updatable	For last updated/created timestamps	<code>TIMESTAMP</code> for <code>last_login</code>
<code>TIME</code>	'-838:59:59' to '838:59:59'	Stores only time (HH:MM:SS)	For durations or specific times	<code>TIME</code> for meeting duration
<code>YEAR</code>	1901 to 2155	Stores a 4-digit year	For year-only values	<code>YEAR</code> for graduation year

How to store JSON data in MySQL

1. Use MySQL JSON Data Type (Best Way)

Starting from MySQL 5.7, MySQL provides a native JSON data type.

👉 Create Table with JSON Column

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  details JSON
);
```

✓ 2. Insert JSON Data

```
INSERT INTO users (name, details)
VALUES (
    'Satya',
    '{"age": 25, "city": "Hyderabad", "skills": ["Java", "Spring Boot"]}'
);
```

✓ JSON must be valid format (double quotes required)

✓ 3. Query JSON Data

👉 Get full JSON

```
SELECT details FROM users;
```

👉 Extract specific field

```
SELECT details->'$.city' AS city FROM users;
```

👉 Extract as text (unquoted)

```
SELECT details->>'$.city' AS city FROM users;
```

✓ 4. Filter Based on JSON Value

```
SELECT * FROM users
WHERE details->>'$.city' = 'Hyderabad';
```

✓ 5. Update JSON Data

```
UPDATE users
SET details = JSON_SET(details, '$.age', 26)
WHERE id = 1;
```

✓ 6. Remove JSON Field

```
UPDATE users
SET details = JSON_REMOVE(details, '$.city')
WHERE id = 1;
```

Table Creation

Definition:

In MySQL, a table is a collection of rows and columns where data is stored in a relational database. Creating a table involves specifying the table's name, the columns, and the data

types of each column. You can also define constraints such as PRIMARY KEY, NOT NULL, DEFAULT, etc., to ensure data integrity and structure.

Syntax:

```
CREATE TABLE table_name (  
    column1 datatype [constraint],  
    column2 datatype [constraint],  
    column3 datatype [constraint],  
    ...  
    [table_constraints]  
);
```

Components:

table_name: Name of the table to be created.

column_name: The name of each column.

datatype: Defines the type of data that the column will hold. (e.g., INT, VARCHAR, DATE, etc.)

constraint: Optional rules that enforce data integrity. Common constraints include:

PRIMARY KEY: Uniquely identifies each record in the table.

NOT NULL: Ensures that the column cannot have NULL values.

DEFAULT: Sets a default value for the column.

UNIQUE: Ensures that all values in a column are unique.

AUTO_INCREMENT: Automatically generates a unique number for the column (usually used for primary keys).

table_constraints: Additional constraints or indexes on the table, like PRIMARY KEY for the whole table.

Example 1: Creating a Basic Table

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50),  
    birth_date DATE  
);
```

This creates a table students with:

student_id: Integer type, it is the primary key (unique and not null).

first_name: A string of up to 50 characters, cannot be NULL.

last_name: A string of up to 50 characters, can be NULL.

birth_date: A date type to store birth dates.

Example 2: Using Auto-Increment for Primary Key

```
CREATE TABLE employees (  
    emp_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    position VARCHAR(50),  
    salary DECIMAL(10, 2)  
);
```

emp_id: The AUTO_INCREMENT feature automatically assigns an incrementing integer to this column whenever a new row is inserted.

Example 3: Creating a Table with Multiple Constraints

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE NOT NULL,  
    total_amount DECIMAL(10, 2) DEFAULT 0.00,  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

The orders table has:

order_id: Auto-incrementing primary key.

customer_id: Foreign key referencing customers table.

order_date: Cannot be NULL.

total_amount: Default value of 0.00 if not provided.

Constraints & Options:

NOT NULL: Ensures that the column must have a value (it cannot be left blank).

DEFAULT: Sets a default value if no value is provided.

UNIQUE: Ensures the values in the column are unique across rows.

AUTO_INCREMENT: Used for auto-generating unique numbers for primary keys.

PRIMARY KEY: Uniquely identifies each row in the table.

FOREIGN KEY: Ensures referential integrity by linking a column in the table to a column in another table.

CHECK: Used to limit the values that can be placed in a column (e.g., CHECK (age >= 18)).

Additional Information:

Data Types:

Numeric Types: INT, BIGINT, FLOAT, DECIMAL.

String Types: VARCHAR, CHAR, TEXT, BLOB.

Date and Time Types: DATE, DATETIME, TIMESTAMP, TIME.

Boolean: BOOLEAN (or TINYINT(1)).

Table Options:

ENGINE: Specifies the storage engine for the table (e.g., InnoDB, MyISAM).

CHARSET: Specifies the character set for the table (e.g., utf8, utf8mb4).

Example with Table Options:

```
CREATE TABLE employees (  
    emp_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    position VARCHAR(50),  
    salary DECIMAL(10, 2)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Common Mistakes:

Forgetting to specify primary keys: Without a primary key, there is no way to uniquely identify rows.

Incorrect Data Types: Always choose the correct data type that matches the kind of data you are storing (e.g., using INT for IDs, VARCHAR for strings).

Not using appropriate constraints: Using constraints like NOT NULL, DEFAULT, UNIQUE, etc., helps enforce better data integrity.

Modifying Tables in MySQL

In MySQL, modifying a table involves making changes to its structure after it has already been created. This can include adding, deleting, or modifying columns, changing table constraints, renaming the table, or even altering its storage engine or character set.

1. ALTER TABLE Statement

The primary command to modify an existing table is ALTER TABLE. The syntax and options for ALTER TABLE vary depending on the modification you want to perform.

General Syntax:

```
ALTER TABLE table_name action;
```

Actions You Can Perform with ALTER TABLE:

1. Add Columns to a Table

You can add one or more columns to a table using the ADD COLUMN keyword.

Syntax:

```
ALTER TABLE table_name ADD COLUMN column_name datatype [constraint];
```

Example:

```
ALTER TABLE employees ADD COLUMN email VARCHAR(100) NOT NULL;
```

Adds a column email to the employees table with a VARCHAR type of length 100 and the constraint NOT NULL.

Add Multiple Columns:

```
ALTER TABLE employees  
ADD COLUMN phone VARCHAR(20),  
ADD COLUMN address VARCHAR(255);
```

2. Modify an Existing Column

To change the data type, name, or constraints of an existing column, you use the MODIFY COLUMN or CHANGE COLUMN keywords.

Syntax for MODIFY COLUMN:

```
ALTER TABLE table_name MODIFY COLUMN column_name new_datatype  
[new_constraint];
```

Syntax for CHANGE COLUMN (renaming a column):

```
ALTER TABLE table_name CHANGE COLUMN old_column_name new_column_name  
new_datatype [new_constraint];
```

Example:

```
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12, 2) NOT NULL;
```

Changes the salary column to DECIMAL(12, 2) type and adds NOT NULL constraint.

Example for Renaming a Column:

```
ALTER TABLE employees CHANGE COLUMN phone phone_number VARCHAR(20);
```

Renames the column phone to phone_number.

3. Drop (Delete) Columns from a Table

You can remove a column using the DROP COLUMN keyword.

Syntax:

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Example:

```
ALTER TABLE employees DROP COLUMN address;
```

Deletes the address column from the employees table.

4. Rename a Table

To rename an existing table, you use the RENAME TO clause.

Syntax:

```
ALTER TABLE old_table_name RENAME TO new_table_name;
```

Example:

```
ALTER TABLE employees RENAME TO staff;
```

Renames the table employees to staff.

5. Change the Table's Storage Engine

You can change the storage engine of a table (e.g., from InnoDB to MyISAM).

Syntax:

```
ALTER TABLE table_name ENGINE = new_engine;
```

Example:

```
ALTER TABLE employees ENGINE = InnoDB;
```

Changes the storage engine of the employees table to InnoDB.

6. Add or Drop Table Constraints

You can add or drop primary keys, foreign keys, and other constraints.

Add Primary Key:

```
ALTER TABLE employees ADD PRIMARY KEY (emp_id);
```

Add Foreign Key:

```
ALTER TABLE orders  
ADD CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES  
customers(customer_id);
```

Drop Primary Key:

```
ALTER TABLE employees DROP PRIMARY KEY;
```

Drop Foreign Key:

```
ALTER TABLE orders DROP FOREIGN KEY fk_customer;
```

7. Change Column Default Value

To change the default value for an existing column, use ALTER COLUMN.

Syntax:

```
ALTER TABLE table_name ALTER COLUMN column_name SET DEFAULT default_value;
```

Example:

```
ALTER TABLE employees ALTER COLUMN salary SET DEFAULT 50000.00;
```

Changes the default value for salary column to 50000.00.

8. Modify Table Character Set or Collation

You can modify the character set or collation of a table.

Syntax:

```
ALTER TABLE table_name CHARACTER SET new_charset COLLATE new_collation;
```

Example:

```
ALTER TABLE employees CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Changes the character set of the employees table to utf8mb4 and the collation to utf8mb4_unicode_ci.

Examples of Common Table Modifications:

Example 1: Adding a New Column

```
ALTER TABLE products ADD COLUMN stock_quantity INT DEFAULT 0;
```

Adds a column `stock_quantity` of type `INT` with a default value of 0.

Example 2: Modifying a Column

```
ALTER TABLE employees MODIFY COLUMN name VARCHAR(255);
```

Modifies the `name` column to be of type `VARCHAR(255)`.

Example 3: Renaming a Column

```
ALTER TABLE products CHANGE COLUMN description product_description TEXT;
```

Renames the `description` column to `product_description`.

Example 4: Removing a Column

```
ALTER TABLE orders DROP COLUMN discount;
```

Removes the `discount` column from the `orders` table.

Example 5: Changing Table Engine

```
ALTER TABLE products ENGINE = InnoDB;
```

Changes the storage engine of the `products` table to `InnoDB`.

Example 6: Adding a Foreign Key

```
ALTER TABLE orders  
ADD CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES  
customers(customer_id);
```

Adds a foreign key on the `customer_id` column that references the `customer_id` column in the `customers` table.

Important Considerations:

Performance Impact: Large tables can take a long time to alter, especially when adding or modifying columns. Always consider the performance impact and plan changes during low-traffic periods if applicable.

Backup: Before performing structural changes to a table (especially for production databases), it's a good practice to back up your data.

Constraints: Be mindful of any existing data when altering constraints. For example, adding a `NOT NULL` constraint to a column that contains `NULL` values will fail.

Locking: Some `ALTER TABLE` operations can lock the table, which may affect database performance during the operation.

📁 Table: MySQL ALTER, DROP, TRUNCATE

Feature / Operation	ALTER TABLE	DROP TABLE	TRUNCATE TABLE
🔧 Changes table structure	✅ Yes	❌ No (deletes the table)	❌ No
🗑️ Deletes table	❌ No	✅ Yes	❌ No
🔪 Deletes all rows	❌ No	✅ Yes (indirectly)	✅ Yes
📄 Keeps table for future use	✅ Yes	❌ No	✅ Yes
🔄 Can be undone (ROLLBACK)	✅ Yes (if in transaction)	❌ No	❌ No (non-transactional)
🔒 Affected by foreign keys	✅ Yes	✅ Yes	✅ Yes (fails if FK constraints)
🚀 Performance (deleting rows)	❌ Slower for many rows	❌ Not applicable	✅ Fast
🔄 Auto-increment reset	❌ No	✅ Yes (table gone)	✅ Yes (resets to 1)
📄 Affects table definition	✅ Yes (add/drop/modify columns)	✅ Yes (removes definition)	❌ No
📄 Transaction-safe	✅ Yes (with InnoDB)	❌ No	❌ No
🔑 Requires privileges	ALTER privilege	DROP privilege	DROP or DELETE privilege

Notes:

- ALTER TABLE is used for structural changes like adding/removing columns, changing data types, renaming columns, adding indexes, etc.
- DROP TABLE completely removes the table and its data from the database.
- TRUNCATE TABLE is used to quickly delete all rows in a table but retain the structure for future use.
- TRUNCATE is not transaction-safe in MySQL and cannot be rolled back, even in InnoDB.
- DROP removes any dependent objects like foreign key constraints unless handled explicitly.
- Use caution with DROP and TRUNCATE in production environments.

CRUD Operations – SQL Basics

What is CRUD?

CRUD stands for Create, Read, Update, Delete — the four basic operations for managing data in a database.

✅ 1. Create

Definition: Add new data to the database.

SQL Command: INSERT

```
INSERT INTO students (id, name, age)
VALUES (1, 'Alice', 20);
```

2. Read

Definition: View or retrieve data.

SQL Command: SELECT

```
SELECT * FROM students;
```

3. Update

Definition: Change existing data.

SQL Command: UPDATE

```
UPDATE students
SET age = 21
WHERE id = 1;
```

4. Delete

Definition: Remove data.

SQL Command: DELETE

```
DELETE FROM students
WHERE id = 1;
```

Summary

Operation	Action	SQL Keyword
Create	Add data	INSERT
Read	View data	SELECT
Update	Change data	UPDATE
Delete	Remove data	DELETE

SQL Data Manipulation Sequence

This note covers the basic steps to manipulate data in SQL, in order:

Insert → Retrieve → Filter → Sort → Update → Delete

1 Inserting Data

Definition: Add new records to a table.

Command: INSERT INTO

```
INSERT INTO students (id, name, age)
VALUES (1, 'Alice', 20);
```

2 Retrieving Data

Definition: View or fetch data from a table.

Command: SELECT

```
SELECT * FROM students;
```

3 Filtering Data

Definition: Get specific data using conditions.

Command: WHERE

```
SELECT * FROM students
WHERE age > 18;
```

4 Sorting Data

Definition: Order the data (ascending or descending).

Command: ORDER BY

```
SELECT * FROM students
ORDER BY age DESC;
```

5 Updating Data

Definition: Modify existing records.

Command: UPDATE + SET

```
UPDATE students
SET age = 21
WHERE id = 1;
```

6 Deleting Data

Definition: Remove records from a table.

Command: DELETE FROM

```
DELETE FROM students
```

```
WHERE id = 1;
```

✓ Quick Summary Table

Step	Action	SQL Keyword
1. Insert	Add data	INSERT INTO
2. Retrieve	View data	SELECT
3. Filter	Condition-based	WHERE
4. Sort	Arrange data	ORDER BY
5. Update	Modify data	UPDATE ... SET
6. Delete	Remove data	DELETE FROM

Retrieval of Data in MySQL

SELECT Statement

Used to retrieve data from one or more tables.

```
SELECT column1, column2 FROM table_name;
```

* selects all columns:

```
SELECT * FROM Customers;
```

WHERE Clause

Filters rows based on a condition.

```
SELECT * FROM Customers WHERE City = 'New York';
```

Comparison Operators

Operator	Meaning
=	Equal to
!= / <>	Not equal to
>, <	Greater/Less
>=, <=	Greater/Less or equal
BETWEEN	Between a range
IN (...)	Match any in list

Logical Operators

AND, OR, NOT

```
SELECT * FROM Customers WHERE City = 'New York' AND Age > 25;
```

ORDER BY

Sorts result set.

```
SELECT * FROM Customers ORDER BY Name ASC;
```

ASC = ascending (default)

DESC = descending

LIMIT

Restricts number of rows returned.

```
SELECT * FROM Customers LIMIT 5;
```

DISTINCT

Removes duplicate values.

SELECT DISTINCT City FROM Customers;

Aggregate Functions

Function	Use
COUNT()	Number of rows
SUM()	Total of values
AVG()	Average value
MAX()	Highest value
MIN()	Lowest value

Example:

SELECT COUNT(*) FROM Customers;

Filtering Data in MySQL

WHERE Clause

Filters rows based on a condition.

```
SELECT * FROM Customers WHERE City = 'New York';
```

 Returns all customers from New York.

Comparison Operators

Operator	Meaning	Example
=	Equal to	WHERE Age = 25
<> / !=	Not equal to	WHERE City != 'Dallas'
>	Greater than	WHERE Age > 30
<	Less than	WHERE Age < 18

>=	Greater or equal	WHERE Age >= 21
<=	Less or equal	WHERE Age <= 65

```
SELECT * FROM Customers WHERE Age >= 30;
```

📌 Returns customers who are 30 years or older.

🧠 Logical Operators

Operator	Description	Example
AND	All conditions true	WHERE City = 'NY' AND Age > 25
OR	At least one true	WHERE City = 'NY' OR City = 'LA'
NOT	Reverses condition	WHERE NOT City = 'Chicago'

```
SELECT * FROM Customers
WHERE City = 'New York' AND Age > 25;
```

📌 Returns customers in New York older than 25.

📄 LIKE Operator

Used for pattern matching.

Pattern Meaning

% Any number of characters

_ Single character

```
SELECT * FROM Customers WHERE Name LIKE 'A%';
```

📌 Names starting with 'A'

```
SELECT * FROM Customers WHERE Email LIKE '%@gmail.com';
```

📌 Emails ending with '@gmail.com'

📦 IN Operator

Check if value exists in a list.

```
SELECT * FROM Customers
WHERE City IN ('New York', 'Chicago', 'Dallas');
```

📌 Returns customers from any of the listed cities.

📅 BETWEEN Operator

Check if value is within a range (inclusive).

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN '2023-01-01' AND '2023-12-31';
```

📌 Orders placed in the year 2023.

IS NULL / IS NOT NULL

Checks for NULL values.

```
SELECT * FROM Customers
WHERE Address IS NULL;
```

 Returns customers with no address.

```
SELECT * FROM Customers
WHERE Address IS NOT NULL;
```

 Returns customers who have an address.

Sorting Data in MySQL

ORDER BY Clause

Used to sort the result set by one or more columns.

```
SELECT * FROM Customers
ORDER BY Name;
```

 Sorts customers alphabetically by name (ascending by default).

ASC and DESC

Keyword	Meaning
ASC	Ascending (default)
DESC	Descending

```
SELECT * FROM Customers
ORDER BY Age DESC;
```

 Returns customers sorted by age from highest to lowest.

Sorting by Multiple Columns

```
SELECT * FROM Customers
ORDER BY City ASC, Name DESC;
```

 Sorts first by City (A–Z), then by Name (Z–A) within each city.

ORDER BY with Aliases or Expressions

You can sort by column aliases or calculated fields.

```
SELECT Name, LENGTH(Name) AS NameLength
FROM Customers
ORDER BY NameLength DESC;
```

 Sorts customers by the length of their names.

⚠ ORDER BY Position (Not Recommended)

You can also sort by column position in the SELECT list:

```
SELECT Name, City FROM Customers
ORDER BY 2; -- Sorts by City
```

📌 Works, but not readable or safe if the query changes.

🔧 ORDER BY + LIMIT (Top-N Query)

```
SELECT * FROM Customers
ORDER BY Age DESC
LIMIT 5;
```

📌 Returns top 5 oldest customers.

Updating Data in MySQL

🔧 UPDATE Statement

Used to modify existing records in a table.

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

✅ Basic Example

```
UPDATE Customers
SET City = 'Los Angeles'
WHERE CustomerID = 3;
```

📌 Updates the city of the customer with ID 3 to "Los Angeles".

⚠ Always Use WHERE!

If you omit the WHERE clause, all rows will be updated.

```
UPDATE Customers
SET City = 'Unknown';
```

📌 This will set the city to 'Unknown' for every customer.

🧠 Updating Multiple Columns

UPDATE Customers

```
SET Name = 'John Doe', City = 'Chicago'
WHERE CustomerID = 5;
```

📌 Updates both name and city for customer with ID 5.

 Use with SELECT for Safe Updates

Always check affected rows before updating:

```
SELECT * FROM Customers
WHERE City = 'Dallas';
```

Then:

```
UPDATE Customers
SET City = 'Houston'
WHERE City = 'Dallas';
```

 Using Expressions in Updates

```
UPDATE Customers
SET Age = Age + 1
WHERE City = 'New York';
```

 Increments age by 1 for all customers in New York.

 **Update Based on NULL**

```
UPDATE Customers
SET Address = 'Pending'
WHERE Address IS NULL;
```

 Fills in missing addresses with 'Pending'.

Deleting Data in MySQL

 **DELETE Statement**

Used to remove one or more records from a table.

```
DELETE FROM table_name
WHERE condition;
```

 Basic Example

```
DELETE FROM Customers
WHERE CustomerID = 4;
```

 Deletes the customer with ID 4.

 **Always Use WHERE!**

If you omit the WHERE clause, all rows in the table will be deleted.

```
DELETE FROM Customers;
```

📌 Deletes all customer records. Dangerous!

Deleting with Multiple Conditions

```
DELETE FROM Customers  
WHERE City = 'Chicago' AND Age < 18;
```

📌 Deletes customers from Chicago who are under 18.

Delete Based on NULL

```
DELETE FROM Customers  
WHERE Email IS NULL;
```

📌 Deletes customers without an email address.

Safe Delete: Check First with SELECT

```
SELECT * FROM Customers  
WHERE City = 'Boston';
```

Then:

```
DELETE FROM Customers  
WHERE City = 'Boston';
```

📌 Always verify before running a DELETE.

Cannot Delete from Multiple Tables

Unlike SELECT, DELETE in MySQL cannot target multiple tables unless using JOIN with alias (advanced use case).

AUTO_INCREMENT After DELETE

Deleting rows does not reset the AUTO_INCREMENT value.

```
DELETE FROM Customers WHERE CustomerID = 10;
```

📌 The next inserted ID will still continue from the last highest value.

Examples

DELETE all rows:

```
DELETE FROM Customers;
```

Deletes all rows but keeps the table structure and AUTO_INCREMENT value.

TRUNCATE all rows:

```
TRUNCATE TABLE Customers;
```

Quickly removes all rows and resets AUTO_INCREMENT to 1.

⚠ When to Use Which?

Use DELETE when:

- You want to delete specific rows using conditions.
- You need to activate triggers.
- You want the option to rollback the delete inside transactions.

Use TRUNCATE when:

- You want to delete all rows quickly.
- You want to reset AUTO_INCREMENT.
- You don't need to activate triggers or worry about rollback.

🔪 TRUNCATE vs DELETE in MySQL

Feature	DELETE	TRUNCATE
Purpose	Deletes rows based on condition	Deletes all rows in the table
Syntax	<code>DELETE FROM table WHERE condition;</code>	<code>TRUNCATE TABLE table;</code>
Condition support	Yes, can delete specific rows	No, deletes entire table
Logging	Fully logged (row by row)	Minimal logging, faster
Speed	Slower on large tables	Faster for deleting all rows
AUTO_INCREMENT reset	No — counter is not reset	Yes — counter is reset to 1
Triggers	Activates DELETE triggers	Does not activate DELETE triggers
Transaction support	Can be rolled back if inside transaction	Often can't be rolled back (depends on storage engine)
Locks	Row-level locking	Table-level locking

Aggregating & Grouping Data in MySQL

What is Aggregation?

Aggregation means summarizing data — like counting rows, finding averages, totals, min/max values, etc.

MySQL provides aggregate functions for this purpose.

Common Aggregate Functions

Function	Description	Example
COUNT()	Counts the number of rows	COUNT(*) or COUNT(column)
SUM()	Adds values in a column	SUM(Salary)
AVG()	Calculates average	AVG(Price)
MIN()	Finds minimum value	MIN(Age)
MAX()	Finds maximum value	MAX(Score)

Examples

```
SELECT COUNT(*) FROM Customers;
```

 Total number of customers

```
SELECT AVG(Salary) FROM Employees;
```

 Average salary of all employees

```
SELECT MIN(OrderDate), MAX(OrderDate) FROM Orders;
```

 Earliest and latest order dates

GROUP BY Clause

GROUP BY is used to group rows that have the same values in one or more columns and then perform aggregation on each group.

Syntax

```
SELECT column, AGG_FUNC(column)
FROM table
GROUP BY column;
```

Basic Example

```
SELECT City, COUNT(*) AS TotalCustomers
FROM Customers
GROUP BY City;
```

 Counts how many customers are in each city

Multiple Aggregates

```
SELECT Department, COUNT(*) AS TotalEmployees, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department;
```

 Shows employee count and average salary per department

🧠 GROUP BY Multiple Columns

```
SELECT City, Gender, COUNT(*) AS Total
FROM Customers
GROUP BY City, Gender;
```

📌 Groups by both City and Gender

🔍 Filtering Groups: HAVING Clause

HAVING is used to filter groups (like WHERE, but for grouped results).

```
SELECT City, COUNT(*) AS TotalCustomers
FROM Customers
GROUP BY City
HAVING COUNT(*) > 5;
```

📌 Returns only cities with more than 5 customers

🔄 Difference Between WHERE and HAVING

Clause	Used For	Applies To
WHERE	Filter rows	Before grouping
HAVING	Filter groups	After grouping

Example using both:

```
SELECT City, AVG(Age) AS AvgAge
FROM Customers
WHERE Age > 18
GROUP BY City
HAVING AVG(Age) > 30;
```

📌 Groups only adults and shows cities where average age is above 30

🔧 Real-World Example

Table: Orders

OrderID	CustomerID	TotalAmount	Status
1	101	150.00	Shipped
2	102	250.00	Pending
3	101	120.00	Shipped

```
SELECT CustomerID, SUM(TotalAmount) AS TotalSpent
FROM Orders
GROUP BY CustomerID;
```

📌 Shows how much each customer has spent

Tips

Always include non-aggregated columns in the GROUP BY clause.

Use aliases (AS) for cleaner output.

Use HAVING instead of WHERE to filter after grouping.

Relationships & Joins in MySQL

1. Relationships in MySQL

What are Relationships?

Define how tables relate to each other to organize data efficiently and avoid duplication.

Types of Relationships		
Relationship Type	Description	Example
One-to-One (1:1)	One row in Table A relates to one row in Table B	Person ↔ Passport
One-to-Many (1:N)	One row in Table A relates to many rows in Table B	Customer ↔ Orders
Many-to-Many (M:N)	Many rows in Table A relate to many in Table B (via junction table)	Students ↔ Courses

2. Keys in Relationships

Key Type	Description	Example
Primary Key (PK)	Unique identifier for each row	<code>customer_id</code> in Customers table
Foreign Key (FK)	References PK of another table	<code>customer_id</code> in Orders table

3. Joins in MySQL

What is a Join?

Combines rows from multiple tables based on related columns.

Why use Joins?

To fetch related data spread across tables.

Enables complex queries to answer business questions.

4. Types of Joins

Join Type	Description	Result
INNER JOIN	Returns rows where there's a match in both tables	Only matching rows
LEFT JOIN	Returns all rows from left table + matches from right	All left rows + matching right rows (NULL if no match)
RIGHT JOIN	Returns all rows from right table + matches from left	All right rows + matching left rows (NULL if no match)
FULL JOIN (Not directly in MySQL)	Returns all rows when matched in either table	All rows from both tables (simulate with UNION)

5. Join Syntax

```
SELECT columns
FROM table1
JOIN_TYPE table2
ON table1.column = table2.column;
```

6. Join Examples

Sample Tables

Customers

customer_id (PK)	name
1	Alice
2	Bob
3	Charlie

Orders

order_id (PK)	customer_id (FK)	product
101	1	Laptop
102	2	Phone
103	1	Keyboard

INNER JOIN

```
SELECT Customers.name, Orders.product
FROM Customers
INNER JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

name	product
Alice	Laptop
Bob	Phone
Alice	Keyboard

Only customers with orders.

● LEFT JOIN

```
SELECT Customers.name, Orders.product
FROM Customers
LEFT JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

name	product
Alice	Laptop
Bob	Phone
Alice	Keyboard
Charlie	NULL

All customers, including those without orders.

● RIGHT JOIN

```
SELECT Customers.name, Orders.product
FROM Customers
RIGHT JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

name	product
Alice	Laptop
Bob	Phone
Alice	Keyboard

All orders with their customers.

● FULL JOIN (Simulated)

```
SELECT Customers.name, Orders.product
FROM Customers
LEFT JOIN Orders ON Customers.customer_id = Orders.customer_id

UNION

SELECT Customers.name, Orders.product
FROM Customers
RIGHT JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

All customers and all orders, matched or unmatched.

7. Bonus: Self Join

Join a table to itself to compare rows.

```
SELECT A.employee_name AS Employee, B.employee_name AS Manager
FROM Employees A
LEFT JOIN Employees B ON A.manager_id = B.employee_id;
```

CROSS JOIN ✕

Returns the Cartesian product of rows from both tables.

Every row from Table A is combined with every row from Table B.

Example:

```
SELECT Customers.Name, Products.ProductName
FROM Customers
CROSS JOIN Products;
```

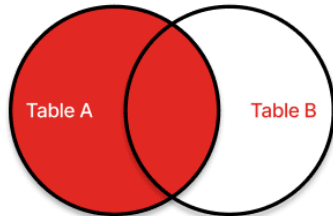
Summary

- Relationships connect tables logically (1:1, 1:N, M:N).
- Keys enforce these connections (Primary and Foreign Keys).
- Joins combine related data across tables.
- Use INNER JOIN for matching data only.
- Use LEFT/RIGHT JOIN to include unmatched rows from one side.
- Simulate FULL JOIN with UNION in MySQL.
- Self Join is useful for hierarchical data.

Concept	Description	Example
One-to-One	One row in A relates to one row B	Person 🧑 — Passport 🛂
One-to-Many	One row in A relates to many rows B	Customer 🛒 — Orders 📦
Many-to-Many	Many rows in A relate to many in B	Students 🎓 — Courses 📖
INNER JOIN	Only matching rows	Customers with orders 🔍
LEFT JOIN	All left rows + matching right	All customers, orders if any 📄
RIGHT JOIN	All right rows + matching left	All orders, customers if any ➡️
FULL JOIN	All rows from both (MySQL via UNION)	Complete match + mismatches 🌿
CROSS JOIN	Cartesian product	Every customer paired with every product ✕

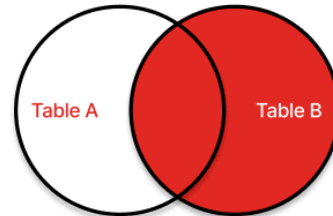
SQL JOINS Visualized with Venn Diagrams

LEFT JOIN



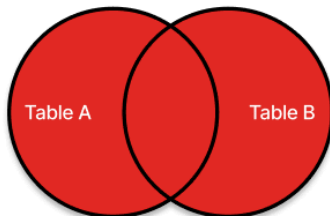
`SELECT * FROM A LEFT JOIN B ON A.key = B.key;`

RIGHT JOIN



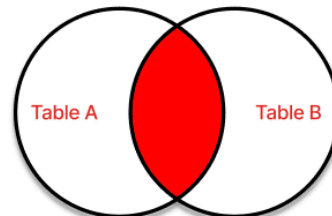
`SELECT * FROM A RIGHT JOIN B ON A.key = B.key;`

FULL OUTER JOIN



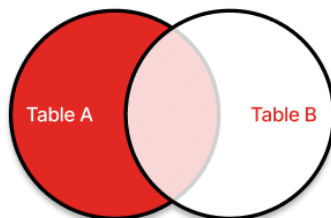
`SELECT * FROM A FULL OUTER JOIN B ON A.key = B.key;`

INNER JOIN



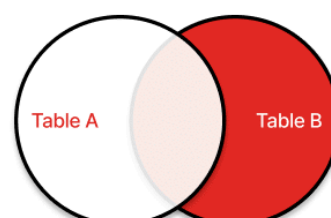
`SELECT * FROM A INNER JOIN B ON A.key = B.key;`

LEFT JOIN excluding INNER JOIN



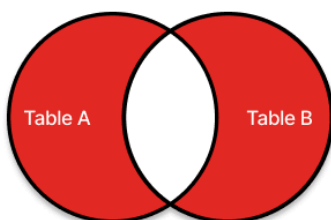
`SELECT * FROM A LEFT JOIN B ON A.key = B.key
WHERE B.key IS NULL;`

RIGHT JOIN excluding INNER JOIN



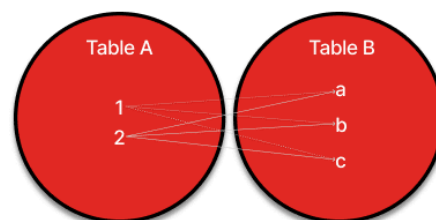
`SELECT * FROM A RIGHT JOIN B ON A.key = B.key
WHERE A.key IS NULL;`

FULL OUTER JOIN excluding INNER JOIN



`SELECT * FROM A FULL OUTER JOIN B ON A.key = B.key
WHERE A.key IS NULL OR B.key IS NULL;`

CROSS JOIN



`SELECT * FROM A CROSS JOIN B;`

Subqueries

1. What is a Subquery?

A Subquery is a query nested inside another query.

It returns data that will be used by the main (outer) query.

Also called: Inner Query or Nested Query.

- ♦ A subquery must be enclosed in parentheses ().

Types of Subqueries:

Type	Description
 Single Row Subquery	Returns only one row
 Multiple Row Subquery	Returns multiple rows
 Multiple Column Subquery	Returns multiple columns
 Correlated Subquery	Depends on the outer query
 Nested Subquery	Subquery inside another subquery

2. Subquery Example (Basic)

-- Get employees who earn more than the average salary

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

- ♦ Here, the inner query gets the average salary.
- ♦ The outer query finds employees who earn more than that.

3. Subquery in SELECT Clause

```
SELECT name,
    (SELECT department_name
     FROM departments
     WHERE departments.id = employees.department_id) AS dept
FROM employees;
```

 Subquery returns department name per row.

📌 4. Subquery in FROM Clause

```
SELECT avg_salary
FROM (
    SELECT AVG(salary) AS avg_salary
    FROM employees
) AS temp;
```

📁 This lets you treat a subquery like a temporary table.

🔗 5. Correlated Subquery

A subquery that uses data from the outer query.

```
SELECT name
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
);
```

🔄 This query compares each employee's salary to the average salary in their department.

✅ 6. Subquery Operators in MySQL 🔑

 Operator	 Use Case	 Description
=	Single Row Subquery	Compare with a single value
IN	Multiple Row Subquery	Match any value in a list
ANY / SOME	One of the values	True if at least one comparison is true
ALL	All values must match	True only if the condition is true for all
EXISTS	Check existence	Returns true if subquery returns any rows

🟢 = Operator (Single Row Subquery)

📌 Use Case: Compare with one value only.

```
SELECT name
FROM employees
WHERE department_id = (
    SELECT id
    FROM departments
    WHERE name = 'Sales'
);
```

 The subquery must return exactly one row, or it will throw an error.

IN Operator (Multiple Row Subquery)

 Use Case: Compare with any value in a list.

```
SELECT name
FROM employees
WHERE department_id IN (
    SELECT id
    FROM departments
    WHERE location = 'New York'
);
```

 Returns employees in any of the departments located in New York.

ANY / SOME Operator

 Use Case: Compare with any value returned by the subquery.

```
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE department_id = 3
);
```

 This returns employees whose salary is greater than at least one person in department 3.

 **SOME** is a synonym of **ANY** — they work the same.

ALL Operator

 Use Case: Compare with all values returned by the subquery.

```
SELECT name, salary
FROM employees
WHERE salary > ALL (
    SELECT salary
    FROM employees
    WHERE department_id = 3
);
```

 Returns employees whose salary is greater than everyone in department 3.

EXISTS Operator

 Use Case: Check if subquery returns any rows.

```
SELECT name
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM departments d
    WHERE d.id = e.department_id
    AND d.location = 'London'
);
```

 Returns employees who belong to departments in London.

SELECT 1 is a dummy value – it doesn't matter what you select in EXISTS.

Only checks if any row exists, not what it contains.

 Quick Recap Table		
 Operator	 Use When...	 Returns True If...
=	You expect one value	The value matches
IN	You expect many values	Value is in the list
ANY / SOME	You want any match	One comparison is true
ALL	You want all to match	All comparisons are true
EXISTS	You want to check existence	Subquery returns rows

Views

A View is a virtual table based on a SELECT query.

It doesn't store data physically.

You can query it like a real table.

Used to simplify complex queries.

 Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table
```

```
WHERE condition;
```

8. View Example

```
CREATE VIEW high_salary_employees AS
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

-- Now you can query the view:

```
SELECT * FROM high_salary_employees;
```

 Easier to read and use!

9. Updating Views

If a view is updatable, you can do:

```
UPDATE view_name
SET column = value
WHERE condition;
```

 Limitations:

Views with GROUP BY, DISTINCT, or joins may not be updatable.

10. Dropping a View

```
DROP VIEW view_name;
```

 Deletes the view (not the data from original tables).

11. Advantages of Views

-  Simplifies complex queries
-  Increases security (hide sensitive columns)
-  Logical data independence
-  Reusability

12. Limitations of Views

-  Can't always be updated
-  No indexes (views don't store data)
-  Dependent on base tables — if table is dropped, view breaks

🧠 13. Common Subquery vs View Usage

Task	Use Subquery or View?
One-time filtering	🟢 Subquery
Reusable reporting	🟡 View
Complex logic simplification	🟡 View
Performance tuning	🟢 Subquery (sometimes faster)

💡 Tips & Best Practices

- 🧠 Use aliases in subqueries for readability.
- 🗑️ Clean up views not in use (DROP VIEW).
- ✅ Use views for consistent business logic.

🚀 Avoid using views with heavy joins in performance-critical apps.

📊 Summary Table		
Concept	Description	Example
🌱 Subquery	Query inside another query	<code>SELECT * FROM ... WHERE col IN (SELECT ...)</code>
📄 View	Virtual table from a SELECT	<code>CREATE VIEW myview AS SELECT ...</code>
🔗 Correlated Subquery	Uses outer query values	<code>WHERE col = (SELECT ... WHERE outer.col = inner.col)</code>

Normalization

Overview

Normalization is the process of organizing the data and the attributes of a database. It is performed to reduce the data redundancy in a database and to make sure that data is stored logically. Data redundancy means having the same data but at different places. It is necessary to remove data redundancy because it causes anomalies in a database which makes it very hard for a database administrator to maintain it.

Why Do We Need Normalization?

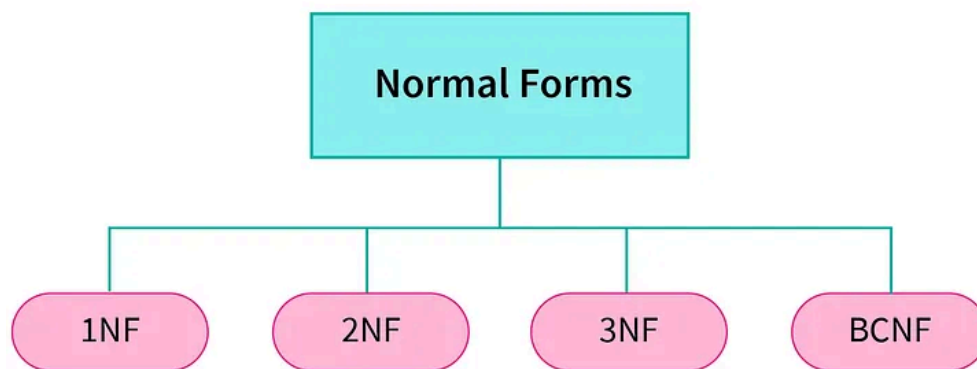
As we have discussed above, normalization is used to reduce data redundancy. It provides a method to remove the following anomalies from the database and bring it to a more consistent state:

A database anomaly is a flaw in the database that occurs because of poor planning and redundancy.

- **Insertion anomalies:** This occurs when we are not able to insert data into a database because some attributes may be missing at the time of insertion.
- **Update anomalies:** This occurs when the same data items are repeated with the same values and are not linked to each other.
- **Deletion anomalies:** This occurs when deleting one part of the data deletes the other necessary information from the database.

Normal Forms

There are four types of normal forms that are usually used in relational databases as you can see in the following figure:



1NF: A relation is in 1NF if all its attributes have an atomic value.

2NF: A relation is in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the candidate key.

3NF: A relation is in 3NF if it is in 2NF and there is no transitive dependency.

BCNF: A relation is in BCNF if it is in 3NF and for every Functional Dependency, LHS is the super key.

To understand the above-mentioned normal forms, we first need to have an understanding of the functional dependencies.

Functional dependency is a relationship that exists between two sets of attributes of a relational table where one set of attributes can determine the value of the other set of attributes. It is denoted by $X \rightarrow Y$, where X is called a determinant and Y is called dependent.

There are various levels of normalizations. Let's go through them one by one:

First Normal Form (1NF)

A relation is in 1NF if every attribute is a single-valued attribute or it does not contain any multi-valued or composite attribute i.e. every attribute is an atomic attribute. If there is a composite or multi-valued attribute, it violates the 1NF. To solve this, we can create a new row for each of the values of the multi-valued attribute to convert the table into the 1NF.

Let's take an example of a relational table <EmployeeDetail> that contains the details of the employees of the company.

Employee Code	Employee Name	Employee Phone Number
101	John	98765623,998234123
101	John	89023467
102	Ryan	76213908
103	Stephanie	98132452

<EmployeeDetail>

Here, the Employee Phone Number is a multi-valued attribute. So, this relation is not in 1NF.

To convert this table into 1NF, we make new rows with each Employee Phone Number as a new row as shown below:

Employee Code	Employee Name	Employee Phone Number
101	John	998234123
101	John	98765623
101	John	89023467
102	Ryan	76213908
103	Stephanie	98132452

<EmployeeDetail>

Second Normal Form (2NF)

The normalization of 1NF relations to 2NF involves the elimination of partial dependencies. A partial dependency exists when any of the non-prime attributes i.e an attribute not a part of the candidate key, is not fully functionally dependent on one of the candidate keys.

For a relational table to be in second normal form, it must satisfy the following rules:

The table must be in first normal form

- It must not contain any partial dependency, i.e., all non-prime attributes are fully functionally dependent on the primary key.
- If a partial dependency exists, we can divide the table to remove the partially dependent attributes and move them to some other table where they fit in well.

Let us take an example of the following <EmployeeProjectDetail> table to understand what is partial dependency and how to normalize the table to the second normal form:

Employee Code	Project ID	Employee Name	Project Name
101	P03	John	Project103
101	P01	John	Project101
102	P04	Ryan	Project104
103	P02	Stephanie	Project102

<EmployeeProjectDetail>

In the above table, the prime attributes of the table are Employee Code and Project ID. We have partial dependencies in this table because Employee Name can be determined by Employee Code and Project Name can be determined by Project ID. Thus, the above relational table violates the rule of 2NF.

To remove partial dependencies from this table and normalize it into second normal form, we can decompose the <EmployeeProjectDetail> table into the following three tables:

Employee Code	Employee Name
101	John
101	John
102	Ryan
103	Stephanie

<EmployeeDetail>

Employee Code	Project ID
101	P03
101	P01
102	P04
103	P02

<EmployeeProject>

Project ID	Project Name
P03	Project103
P01	Project101
P04	Project104
P02	Project102

<ProjectDetail>

Thus, we've converted the <EmployeeProjectDetail> table into 2NF by decomposing it into <EmployeeDetail>, <ProjectDetail> and <EmployeeProject> tables. As you can see, the above tables satisfy the following two rules of 2NF as they are in 1NF and every non-prime attribute is fully dependent on the primary key.

Get Yash Goel's stories in your inbox
Join Medium for free to get updates from this writer.

Enter your email
Subscribe

Remember me for faster sign in

The relations in 2NF are clearly less redundant than relations in 1NF. However, the decomposed relations may still suffer from one or more anomalies due to the transitive dependency. We will remove the transitive dependencies in the Third Normal Form.

Third Normal Form (3NF)

The normalization of 2NF relations to 3NF involves the elimination of transitive dependencies.

A functional dependency $X \rightarrow Z$ is said to be transitive if the following three functional dependencies hold:

$X \rightarrow Y$
 Y does not $\rightarrow X$
 $Y \rightarrow Z$

For a relational table to be in third normal form, it must satisfy the following rules:

The table must be in the second normal form

No non-prime attribute is transitively dependent on the primary key.

For each functional dependency $X \rightarrow Z$ at least one of the following conditions hold:

X is a super key of the table

Z is a prime attribute of the table

If a transitive dependency exists, we can divide the table to remove the transitively dependent attributes and place them to a new table along with a copy of the determinant.

Let us take an example of the following <EmployeeDetail> table to understand what is transitive dependency and how to normalize the table to the third normal form:

Employee Code	Employee Name	Employee Zipcode	Employee City
101	John	110033	Model Town
101	John	110044	Badarpur
102	Ryan	110028	Naraina
103	Stephanie	110064	Hari Nagar

<EmployeeDetail>

The above table is not in 3NF because it has Employee Code $\rightarrow$ Employee City transitive dependency because:

- Employee Code $\rightarrow$ Employee Zipcode
- Employee Zipcode $\rightarrow$ Employee City

Also, Employee Zipcode is not a super key and Employee City is not a prime attribute.

To remove transitive dependency from this table and normalize it into the third normal form, we can decompose the <EmployeeDetail> table into the following two tables:

Employee Code	Employee Name	Employee Zipcode
101	John	110033
101	John	110044
102	Ryan	110028
103	Stephanie	110064

<EmployeeDetail>

Employee Zipcode	Employee City
110033	Model Town
110044	Badarpur
110028	Naraina
110064	Hari Nagar

<EmployeeLocation>

Thus, we've converted the <EmployeeDetail> table into 3NF by decomposing it into <EmployeeDetail> and <EmployeeLocation> tables as they are in 2NF and they don't have any transitive dependency.

The 2NF and 3NF impose some extra conditions on dependencies on candidate keys and remove redundancy caused by that. However, there may still exist some dependencies that cause redundancy in the database. These redundancies are removed by a more strict normal form known as BCNF.

Boyce–Codd Normal Form (BCNF)

Boyce-Codd Normal Form is an advanced version of 3NF as it contains additional constraints compared to 3NF.

For a relational table to be in Boyce-Codd normal form, it must satisfy the following rules:

- The table must be in the third normal form
- For every non-trivial functional dependency $X \rightarrow Y$, X is the superkey of the table. That means X cannot be a non-prime attribute if Y is a prime attribute.

A superkey is a set of one or more attributes that can uniquely identify a row in a database table.

Let us take an example of the following <EmployeeProjectLead> table to understand how to normalize the table to the BCNF:

Employee Code	Project ID	Project Leader
101	P03	Grey
101	P01	Christian
102	P04	Hudson
103	P02	Petro

<EmployeeProjectLead>

The above table satisfies all the normal forms till 3NF, but it violates the rules of BCNF because the candidate key of the above table is {Employee Code, Project ID}. For the non-trivial functional dependency, Project Leader $\rightarrow$ Project ID, Project ID is a prime attribute but Project Leader is a non-prime attribute. This is not allowed in BCNF.

To convert the given table into BCNF, we decompose it into three tables:

Employee Code	Project ID
101	P03
101	P01
102	P04
103	P02

<EmployeeProject>

Project Leader	Project ID
Grey	P03
Christian	P01
Hudson	P04
Petro	P02

<ProjectLead>

Thus, we've converted the <EmployeeProjectLead> table into BCNF by decomposing it into <EmployeeProject> and <ProjectLead> tables.

Normal Form	Description	Benefits	Drawbacks
1NF	Each table has a primary key and each column contains atomic values.	Eliminates duplicate data and ensures data consistency.	Can result in large tables with many columns.
2NF	Each non-key column is dependent on the primary key.	Reduces data redundancy and simplifies data management.	Can result in complex table relationships and slower queries.
3NF	Each non-key column is not transitively dependent on the primary key.	Further reduces data redundancy and improves data consistency.	Can result in even more complex table relationships and slower queries.
BCNF	Every determinant in the table is a candidate key.	Eliminates the possibility of functional dependencies between non-key attributes.	Can be difficult to achieve and may result in large numbers of tables.
4NF	Eliminates the possibility of multi-valued dependencies in a table.	Reduces data redundancy and simplifies data management.	Can result in large numbers of tables and slower queries.
5NF	Handles complex many-to-many relationships in a database.	Improves data consistency and simplifies data management.	Can result in a large number of tables and slower queries.

Stored Procedures

A Stored Procedure is a group of SQL statements stored in the database that can be executed as a single unit. Think of it as a function in programming — ideal for performing complex or repetitive operations.

When to Use:

- When you have business logic that needs to be reused.
- To reduce redundant code across your application.
- To improve performance and security by reducing client-server communication.

Example:

```
CREATE PROCEDURE AddNewUser (
    IN name VARCHAR(100),
    IN email VARCHAR(100)
)
BEGIN
```

```
INSERT INTO users (name, email)
VALUES (name, email);
END;
```

How to Call It:

```
CALL AddNewUser('Jane Doe', 'jane@example.com');
```

Functions

A Function in SQL is a routine that takes input and returns a single value. Unlike stored procedures, functions are designed to be used within SQL expressions, especially in SELECT queries.

When to Use:

- When you need to return a calculated value.
- To encapsulate logic that should be used inside SQL statements.

Example:

```
CREATE FUNCTION TotalPrice(price DECIMAL(10,2), qty INT)
RETURNS DECIMAL(10,2)
BEGIN
    RETURN price * qty;
END;
```

Usage

```
SELECT product_name, TotalPrice(unit_price, quantity) AS total_price
FROM order_items;
```

Triggers

A Trigger is a special type of stored procedure that is automatically executed when a specific database event occurs — such as an INSERT, UPDATE, or DELETE.

Benefits:

- Helps maintain data integrity automatically.
- Enables automatic logging or auditing.
- Reduces application-side logic and increases consistency.

Example

```
CREATE TRIGGER LogInsertUser
AFTER INSERT ON users
FOR EACH ROW
BEGIN
    INSERT INTO user_logs (user_id, action, log_time)
    VALUES (NEW.id, 'INSERT', NOW());
END;
```

This trigger automatically creates a log entry whenever a new user is added to the users table.

Best Practices

- **Avoid overly complex triggers:** Keep logic inside triggers lightweight to prevent performance bottlenecks.
- **Use clear naming conventions:** For example, prefix with sp_ for stored procedures, fn_ for functions, and trg_ for triggers.
- **Centralize business logic:** Keep reusable logic within procedures or functions instead of duplicating it in your application.
- **Test thoroughly before deployment:** Changes in stored logic can affect your entire system — test locally or in staging environments first.
- **Real-World Implementation:** Online Bookstore Case Study

To truly understand how stored procedures, functions, and triggers can be used in practice, let's explore a real-world scenario: **an online bookstore that manages its inventory, customer purchases, and membership statuses.**

In this case, we'll use a **SQL Function to calculate discounts**, a **Stored Procedure to manage transactions**, and a **Trigger to automate membership upgrades**. Here's how each component plays a role in making the system smarter and more efficient.

SQL Function: Calculating Customer Discounts

When customers make purchases, the system calculates a discount based on their total spending. This logic is encapsulated in a SQL function:

```
DELIMITER $$

CREATE FUNCTION calculate_discount(total_spent DECIMAL(10,2))
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
```

```

DECLARE discount DECIMAL(5,2);

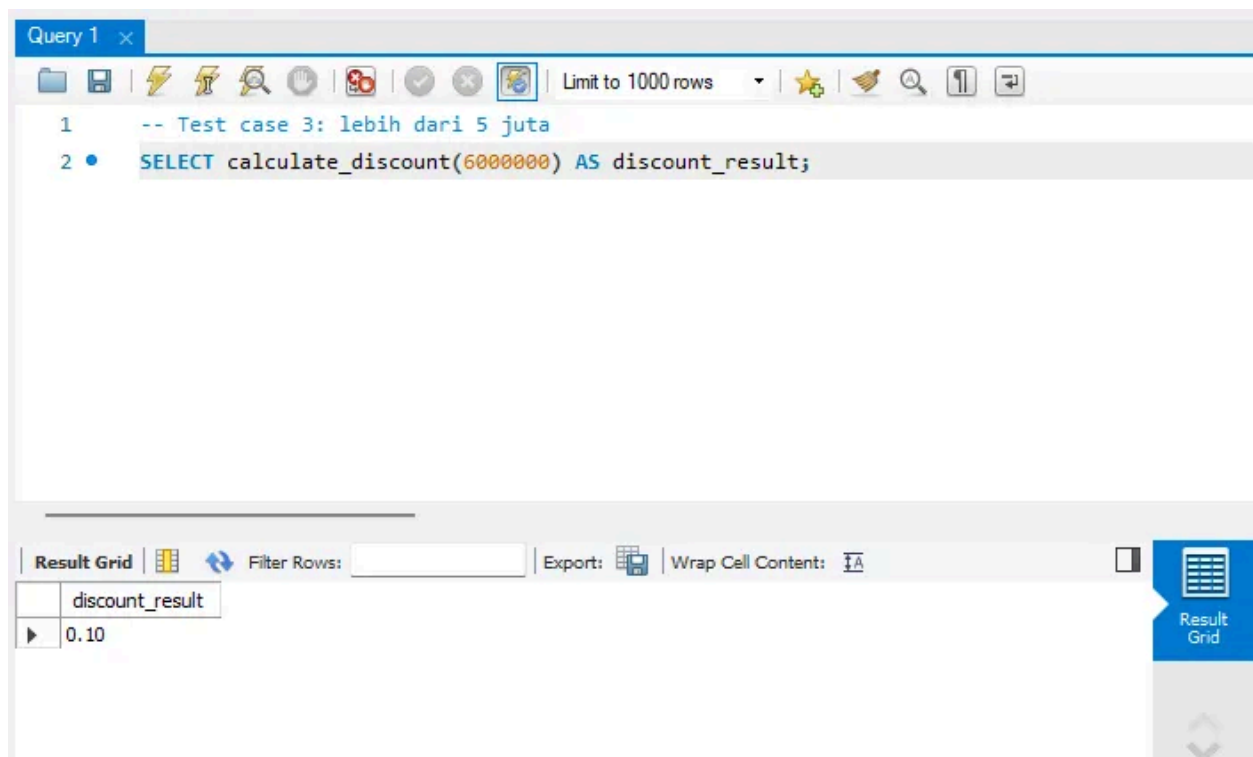
IF total_spent < 1000000 THEN
    SET discount = 0;
ELSEIF total_spent < 5000000 THEN
    SET discount = 0.05;
ELSE
    SET discount = 0.10;
END IF;

RETURN discount;
END$$

DELIMITER ;

```

This function ensures that discount logic remains centralized and reusable wherever needed — making calculations consistent across the system.



The screenshot shows a SQL query editor window titled "Query 1". The query is as follows:

```

1  -- Test case 3: lebih dari 5 juta
2  • SELECT calculate_discount(6000000) AS discount_result;

```

The query is executed, and the result is displayed in a "Result Grid" at the bottom. The grid shows a single row with the column name "discount_result" and the value "0.10".

discount_result
0.10

Stored Procedure: Adding a New Transaction

To handle book purchases, we define a stored procedure that performs several operations in a single call:

Retrieves the price of the selected book

Checks if enough stock is available
Records the transaction
Updates inventory
Increments the customer's total spending
Here's how that logic looks in SQL:

```
DELIMITER $$

CREATE PROCEDURE add_transaction(
  IN p_customer_id INT,
  IN p_book_id INT,
  IN p_quantity INT
)
BEGIN
  DECLARE v_price DECIMAL(10,2);
  DECLARE v_total DECIMAL(10,2);

  SELECT price INTO v_price FROM books WHERE book_id = p_book_id;

  SET v_total = v_price * p_quantity;

  IF (SELECT stock FROM books WHERE book_id = p_book_id) < p_quantity THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock';
  ELSE

    INSERT INTO transactions(customer_id, book_id, amount, total_price,
transaction_date)
      VALUES(p_customer_id, p_book_id, p_quantity, v_total, CURDATE());

    UPDATE books SET stock = stock - p_quantity WHERE book_id = p_book_id;

    UPDATE customers SET order_total = order_total + v_total WHERE
customer_id = p_customer_id;

    SELECT 'Transaction successfully recorded' AS message;
  END IF;
END$$

DELIMITER ;
```

This procedure not only reduces boilerplate logic in your application code but also ensures atomic operations within the database — everything happens together or not at all.

Query 1 x

Limit to 1000 rows

```

1  SELECT
2      c.name AS customer_name,
3      b.title AS book_title,
4      t.amount,
5      t.total_price,
6      b.stock AS remaining_stock,
7      c.order_total
8  FROM
9      transactions t
10 JOIN customers c ON t.customer_id = c.customer_id
11 JOIN books b ON t.book_id = b.book_id

```

Result Grid

	customer_name	book_title	amount	total_price	remaining_stock	order_total
▶	Andi Wijaya	Atomic Habits	2	240000.00	18	6240000.00

Result Grid

Trigger: Auto-Updating Customer Membership Status

To make the system more intelligent, we introduce a trigger that automatically evaluates a customer's total spending every time their data is updated. Based on this value, the customer's membership level is adjusted:

```

DELIMITER $$

CREATE TRIGGER update_membership_status
AFTER UPDATE ON customers
FOR EACH ROW
BEGIN
    DECLARE new_status ENUM('reguler', 'gold', 'platinum');

    IF NEW.order_total >= 5000000 THEN
        SET new_status = 'platinum';
    ELSEIF NEW.order_total >= 1000000 THEN
        SET new_status = 'gold';
    ELSE
        SET new_status = 'reguler';
    END IF;

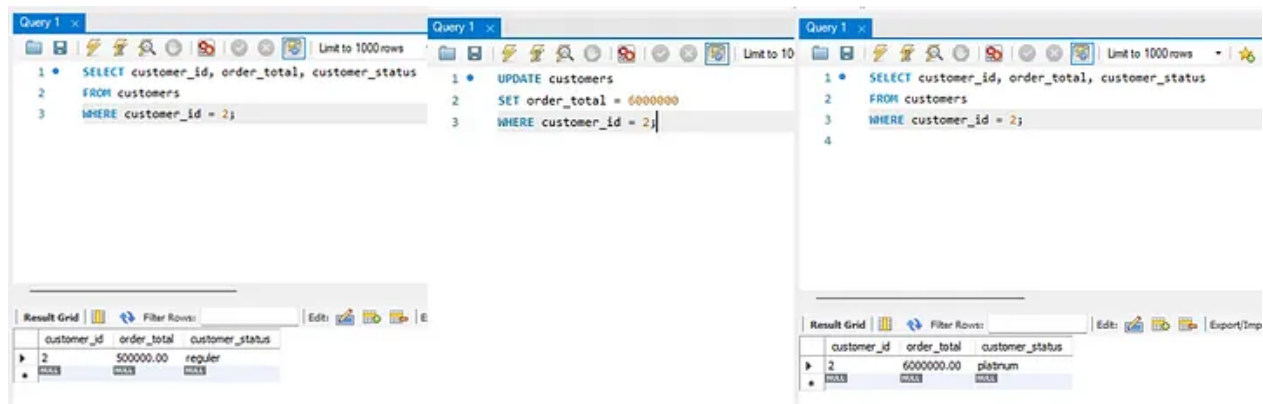
    IF NEW.customer_status != new_status THEN
        UPDATE customers
        SET customer_status = new_status
    END IF;
END IF;

```

```
WHERE customer_id = NEW.customer_id;
END IF;
END$$

DELIMITER ;
```

Thanks to this trigger, customers are automatically upgraded without any manual intervention — an excellent example of how Triggers can streamline business processes and reduce human error.



Conclusion

By mastering Stored Procedures, Functions, and Triggers, you're not just writing SQL — you're building intelligent, efficient, and robust database systems. These features empower your database to do more: automate tasks, enforce business rules, and respond to changes with precision.

A smart database is a powerful ally in building scalable, maintainable applications.

Indexing

In SQL, retrieving data efficiently is crucial for performance, especially when working with large datasets. Without optimization, querying a table can become slow and resource-intensive. This is where indexes play a key role. An index in SQL is a data structure that improves the speed of data retrieval operations in a database. It works like a lookup guide that helps the database find specific rows without scanning the entire table.

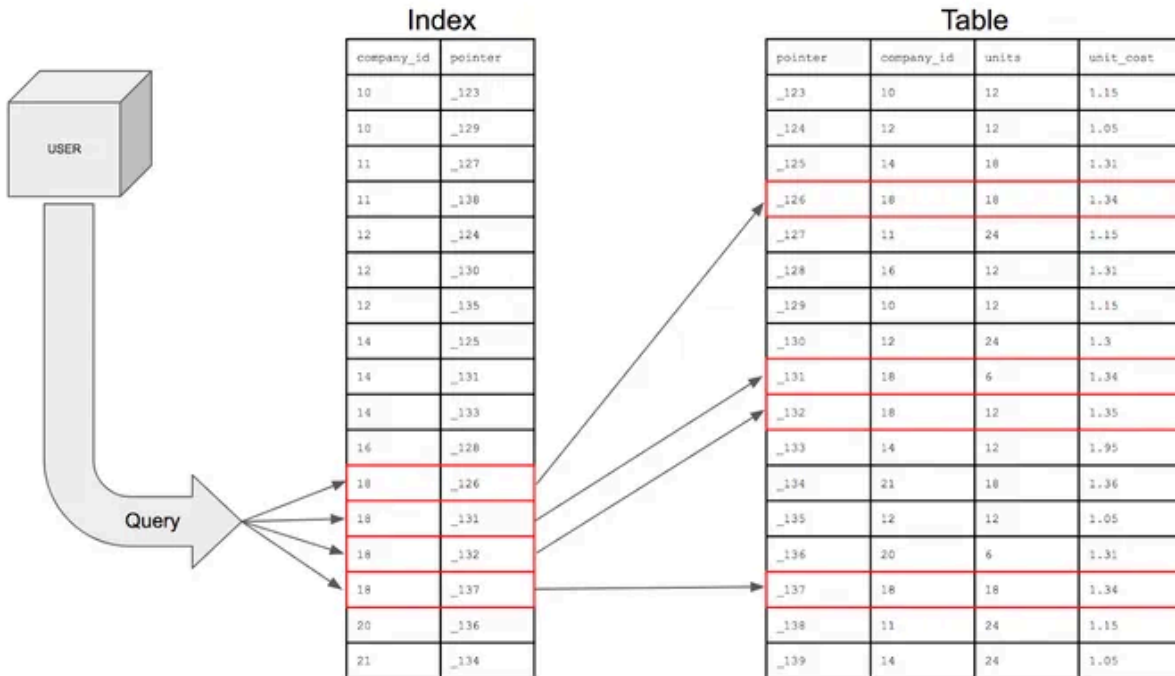
By structuring data intelligently, indexes significantly enhance query execution speed and overall database efficiency.

Index in SQL: Working

Instead of scanning an entire table, SQL uses indexes to quickly find relevant rows, significantly improving query speed.

The image below represents how the index works in the SQL database for faster data retrieval.

How Index in SQL works?



In the above image:

User: Represents a user making a query to the database.

Query: The user's request for specific data.

Index Table (Left Table): In an indexed table, data is organized in two structures:

- **company_id column** → Contains indexed values from the main table.
- **pointer column** → Stores references (pointers) to the actual data rows in the main table. These pointers are typically row identifiers or physical addresses.

Main Table (Right Table):

pointer column: Matches the pointer values in the Index Table, acting as a link between the index and actual data.

company_id, units, unit_cost columns: Represent the actual table data.

Here is the step-by-step process of how the Index works:

STEP 1: User Query: The user submits a query that filters or searches for data based on the `company_id` column. For example, "Find all rows where `company_id` is 18."

STEP 2: Index Lookup: Instead of scanning the entire Table from top to bottom, the database first looks up the `company_id` in the Index table.

STEP 3: Pointer Retrieval: The index quickly finds the matching `company_id` values (in this case, 18) and retrieves the corresponding pointers.

STEP 4: Data Retrieval: Using the pointers, the database directly accesses the relevant rows in the Table. In the image, the red highlighted rows in Table are the rows identified by the pointers found in the Index.

SQL databases commonly implement indexes using B-tree (Balanced Tree) structures, which efficiently handle:

- **Searching** ($O \log N$ complexity).
- **Insertion & Deletion** (Automatically maintains a balanced structure).
- **Range Queries** (Useful for operations like BETWEEN, ORDER BY).

To create an index in SQL, use the following statement:

```
CREATE INDEX idx_company_id ON your_table_name (company_id);
```

This statement creates an index named `idx_company_id` on the `company_id` column of the table `your_table_name`.

Trade-offs

While indexes significantly improve query performance, they come with some trade-offs:

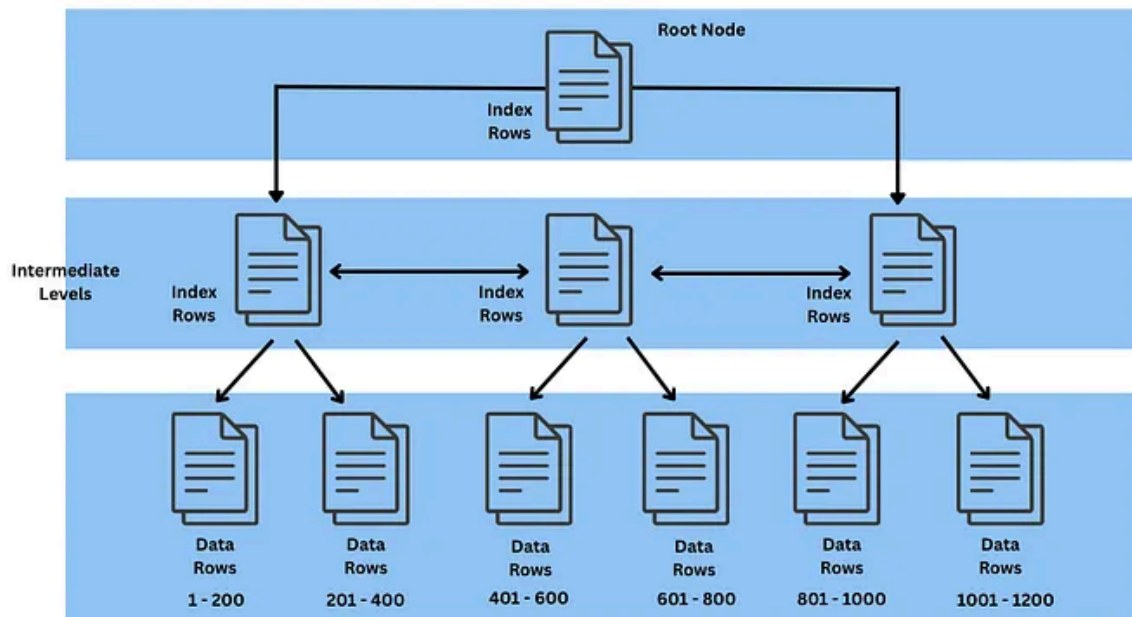
Storage Overhead: Indexes require additional disk space, especially for large tables with multiple indexes.

Maintenance Overhead: Indexes need to be updated whenever data is inserted, updated, or deleted. This can slow down write operations (INSERT, UPDATE, DELETE).

Data Structures Used for Index in SQL

Indexes in SQL databases significantly enhance query performance by organizing data in a structured manner. The most commonly used index structure is the B-tree (Balanced Tree), which allows efficient searching, sorting, and range-based queries.

The below image represents a simplified B-tree index structure used in SQL databases to speed up data retrieval.



How Index in SQL Works?

It shows the following key components:

- **Root Node:** The topmost node of the index. It acts as the starting point for searches.
- **Intermediate Levels:** These are the nodes between the root and the leaf nodes. They guide the search towards the correct leaf node.
- **Data Pages (Leaf Nodes):** These are the bottom-most nodes and contain pointers to the actual data rows in the database. In this example, each data page stores a range of data (e.g., 1–200, 201–400, etc.).
- **Index Rows:** Each node in the index (root and intermediate levels) contains “Index Rows.” These rows essentially hold pointers and key values that help navigate the index.

When an index is created on a column (or multiple columns), the database maintains an organized copy of the indexed data in a B-tree structure. This significantly reduces the time required to locate specific data compared to scanning the entire table.

Here's how:

- **Start at the Root Node:** The database begins its search at the root node.
- **Navigate Through Intermediate Levels:** It compares the search value to the index rows in the root node and follows the pointer to the appropriate child node in the next level. This process repeats until it reaches a leaf node.
- **Locate the Data Page:** The leaf node contains pointers to the actual data pages (or rows) that match the search criteria.

- **Retrieve the Data:** The database then fetches the data directly from the data pages, avoiding a full table scan.

Example Query Execution with Index

Consider a Customers table with an index on the CustomerID column. Suppose you execute the following query:

```
SELECT * FROM Customers WHERE CustomerID = 550;
```

The database would use the index to efficiently find the row(s) with CustomerID = 550 by:

- Starting at the root node.
- Navigating through the intermediate levels based on the index rows.
- Reaching the leaf node that contains the range 401–600.
- Accessing the data page within that range to retrieve the row with CustomerID = 550.

Benefits of B-tree Indexing

- **Logarithmic Search Time ($O \log N$):** The hierarchical structure ensures rapid lookups.
- **Balanced Tree Property:** Nodes remain balanced, preventing performance degradation.
- **Efficient Range Queries:** Useful for queries involving BETWEEN, ORDER BY, and GROUP BY operations.

Types of Index in SQL

Clustered Index in SQL

A special type of index that determines the physical storage order of data rows in a table. It directly organizes the data itself, making the leaf nodes of the index the actual data rows.

The data rows in the table are physically stored in the order defined by the clustered index key. When a query uses the clustered index key, the database traverses the B-tree to find the leaf node (data row) directly. Range queries are very efficient as the data is physically contiguous.

Use Cases: Ideal for columns frequently used in range queries, ORDER BY, or GROUP BY clauses. Primary keys are often used as clustered indexes.

Non Clustered Index in SQL

An index that is stored separately from the data rows. It contains pointers (row locators) to the actual data, allowing for faster lookups without altering the physical storage order of the table.

The leaf nodes of the non-clustered index contain the indexed key values and pointers (row locators) to the actual data rows in the table.

The database searches the B-tree index to find the pointers. Then, it uses these pointers to retrieve the data rows from their physical locations.

Use Cases: Useful for columns frequently used in WHERE clauses but not used in sorting or grouping. You can have multiple non-clustered indexes per table.

Here is the difference between clustered and non clustered Index in SQL:

Feature	Clustered Index	Non-Clustered Index
Physical Order	Determines the physical order of data rows.	Does not affect the physical order of data.
Number Allowed	Only one per table.	Multiple per table allowed.
Leaf Nodes	Actual data rows.	Pointers (row locators) to data rows.
Storage	Data rows are stored in index order.	Stored separately from the data rows.
Range Queries	Very efficient for range queries.	Less efficient for range queries.
Retrieval Speed	Faster for retrieving entire rows.	Slower due to pointer lookup for data.
Common Use	Primary key, frequently used columns in range queries.	Columns used in WHERE clauses, foreign keys.

Difference between Clustered and non clustered index in SQL

B-tree Indexes (Balanced Tree)

A self-balancing tree data structure is used to efficiently organize and retrieve data. It maintains sorted data and allows for logarithmic time complexity for search, insertion, and deletion operations.

The database navigates the tree by comparing the search value with the key values in each node, following the appropriate pointers. The leaf nodes contain either the data rows (clustered index) or pointers to the data rows (non-clustered index).

Use Cases: The default index type in many database systems, suitable for a wide range of queries.

Hash Indexes

An index that uses a hash function to map index keys to their corresponding data locations. It provides very fast lookups for equality comparisons but is not suitable for range queries.

A hash function is applied to the indexed value, producing a hash code. The hash code is used to determine the location of the data in the index. Queries that use equality comparisons (e.g., WHERE column = value) can quickly retrieve data using the hash index.

Use Cases: Hash indexes in SQL are best used for extremely fast equality lookups on columns where range queries are never needed.

Composite Indexes (Multi-Column Index)

An index built on two or more columns of a table. It optimizes queries that filter or sort based on the combined values of those columns, with the order of columns being significant.

The index is built on the combined values of the specified columns, forming a composite key.

The index is stored in sorted order based on the composite key. Queries that match all or a leading subset of the columns in the composite key can efficiently use the index.

Use Cases: Useful for queries that filter or sort data based on multiple columns.

Unique Indexes

An index that enforces uniqueness constraints on the indexed column(s). It ensures that no two rows have the same value(s) in the indexed column(s).

The database ensures that no two rows have the same value for the indexed column(s). When inserting or updating data, the database uses the index to quickly check for duplicate values.

For queries that search for a specific unique value, the index allows for fast retrieval.

Use Cases: Enforcing data integrity, preventing duplicate values in columns like email addresses or usernames.

Full-Text Indexes

A specialized index is used for efficient searching of text data within character-based columns. It breaks down text into tokens and allows for keyword and phrase searches.

Text data is broken down into individual words or tokens. An inverted index is created, mapping each token to the documents or rows that contain it.

When a full-text search is performed, the database uses the inverted index to find the relevant documents or rows.

Use Cases: Implementing search functionality for documents, articles, or product descriptions.

Spatial Indexes

An index designed for spatial data types (e.g., geometry, geography). It optimizes spatial queries by organizing data based on its location and shape.

Spatial data is divided into a grid or hierarchical structure. Spatial objects are stored in the appropriate cells or regions of the index.

Spatial queries (e.g., finding objects within a distance) can efficiently use the index to narrow down the search space.

Use Cases: Applications involving geographic information systems (GIS), location-based services, or mapping.

Filtered Indexes

A non-clustered index that includes only a subset of rows from a table, defined by a filter condition. It reduces index size and improves performance for filtered queries.

The index includes only a subset of rows that meet a specific filter condition. Queries that match the filter condition can use the index to access only the relevant rows.

Use Cases: Optimizing queries that frequently filter data based on specific criteria, such as active customers or recent orders.

Column Store Index

An index that stores data in columnar format, optimizing analytical queries that access a small subset of columns. It provides high compression and efficient aggregation.

Data is stored in columns rather than rows. Queries can efficiently access and process only the required columns. Columnar storage allows for high compression ratios, reducing storage space and I/O.

Use Cases: Data warehousing and analytical workloads that involve aggregating or summarizing large datasets.

Bitmap Index

An index that uses bitmaps to represent the presence of values in a column. It is efficient for columns with low cardinality and for queries with multiple AND or OR conditions.

Each distinct value in the indexed column is represented by a bitmap. Queries can efficiently combine bitmaps using bitwise operations (AND, OR, NOT). It is suitable for columns with a small number of distinct values.

Use Cases: Columns with low cardinality (few distinct values), such as gender or status.

Reverse Index

An index that stores the reversed values of the indexed column. It optimizes queries that use trailing wildcard pattern matching with the LIKE operator.

The index stores the reversed values of the indexed column. Queries with trailing wildcards (e.g., LIKE '%abc') can use the index to efficiently search for patterns at the end of strings.

Use Cases: Optimizing queries that use the LIKE operator with leading wildcards (e.g., LIKE '%abc').

How to Create an Index in SQL?

Creating indexes in SQL is done using the CREATE INDEX statement. The syntax can vary slightly depending on the specific SQL database system you're using (e.g., MySQL, PostgreSQL, SQL Server, Oracle), but the general principles remain the same.

```
CREATE [UNIQUE] INDEX index_name  
ON table_name (column1, column2, ...);
```

In the above syntax:

- **CREATE INDEX** command will create a new index.
- **[UNIQUE]** is an optional command that specifies that the index should enforce uniqueness on the indexed column(s). If used, it prevents duplicate values in the indexed columns.
- **index_name** command is used to give the name to the index. You must give a descriptive name as per the table data.
- **table_name** is the name of the table on which you want to create the index.
- **(column1, column2, ...)** are the columns that are added to the index. You can specify one or more columns. If you specify multiple columns, it becomes a composite index.

Syntax for a Simple Index:

```
CREATE INDEX idx_lastname  
ON Customers (LastName);  
Syntax for a Unique Index:
```

```
CREATE UNIQUE INDEX idx_email  
ON Users (Email);
```

This creates a unique index named idx_email on the Email column of the Users table. This ensures that no two users can have the same email address.

Syntax for a Composite Index:

```
CREATE INDEX idx_city_lastname  
ON Customers (City, LastName);
```

This creates a composite index named `idx_city_lastname` on the `City` and `LastName` columns of the `Customers` table. This is useful for queries that filter or sort based on both city and last name.

Syntax for a Clustered Index:

```
CREATE CLUSTERED INDEX idx_customerid  
ON Customers (CustomerID);
```

In SQL Server, you can explicitly create a clustered index. In many other database systems, the primary key automatically creates a clustered index.

Syntax for a Filtered Index:

```
CREATE INDEX idx_active_orders  
ON Orders (OrderDate)  
WHERE IsActive = 1;
```

This SQL Server example creates an index on the `OrderDate` column only for rows where `IsActive` is 1.

Rebuilding and Reorganizing the Indexes in SQL

To maintain optimal performance, SQL databases provide two key maintenance operations:

- Rebuilding Indexes in SQL
- Reorganizing Indexes in SQL

Rebuilding the Indexes in SQL

Rebuilding an index recreates it from scratch, eliminating fragmentation and optimizing its structure. This process is beneficial when an index becomes highly fragmented or corrupted, causing performance issues.

MySQL:

```
ALTER TABLE Customers DROP INDEX idx_lastname, ADD INDEX idx_lastname  
(LastName);
```

SQL Server:

```
ALTER INDEX idx_lastname ON Customers REBUILD;
```

Oracle:

```
ALTER INDEX idx_lastname REBUILD;
```

PostgreSQL:

```
REINDEX INDEX idx_lastname;
```

When an index is rebuilt, the database engine drops the existing index and creates a new one using the current table data. This often results in a more compact and efficient index structure, improving query performance.

Reorganizing Indexes

Reorganizing an index is a less intensive operation that addresses fragmentation by physically reordering the leaf nodes of the index to align with logical order. Unlike rebuilding, reorganizing does not drop and recreate the index but instead optimizes its existing structure.

Syntax:

ALTER INDEX idx_lastname ON Customers REORGANIZE;

This command rearranges the index's leaf nodes to improve contiguity, reducing the number of I/O operations needed for data retrieval. Since reorganizing is less resource-intensive than rebuilding, it is a preferred option when the index fragmentation level is moderate.

How to Choose the Right Indexes in SQL?

Selecting the right indexes improves query speed while reducing storage and maintenance overhead.

- **Identify Key Queries** — Focus on queries using WHERE, JOIN, ORDER BY, and GROUP BY.
- **Choose the Right Index Type** — Use primary key, unique, clustered, non-clustered, composite, or full-text indexes based on query needs.
- **Index WHERE & JOIN Columns** — Index columns are frequently used in filtering and join for faster lookups.
- **Avoid Over-Indexing** — Too many indexes slow down INSERT, UPDATE, and DELETE operations.
- **Use Covering Indexes** — Combine multiple columns in one index to speed up complex queries.
- **Check Selectivity** — Indexing unique columns is more effective than low-cardinality ones.
- **Monitor & Optimize** — Regularly review and remove unused indexes to improve performance.

Choosing the right type of index based on query patterns ensures optimal performance for various use cases.

ACID Properties

Pre-requisite

To understand this blog, you should have at least a basic knowledge of SQL. If you can read and understand something like `SELECT name FROM users_table`, you'll be able to follow along easily.

Content of this blog

- What is a transaction
- Introduction to ACID Properties
- Atomicity
- Isolation
- Durability
- Consistency

This is not just a theoretical blog; we will do everything with hands-on tutorials in PostgreSQL.

What is a transaction?

A transaction is a group of SQL statements (such as `SELECT`, `UPDATE`, `INSERT`, etc.) that are treated as a single unit of work within a database.

In a transaction, you might execute five SQL statements. Either all five statements complete successfully, or if any one of them fails, the entire transaction fails, and none of the changes are applied.

If you are reading this for the first time, you might be wondering why we write 4 SQL statements as a single unit. Why can't we write those 4 SQL statements individually? Well, there are many scenarios where you want to treat a bunch of SQL statements as a single unit: either all of them are executed successfully, or if one fails for some reason, none of them are executed.

Some examples of transactions may be:

E-commerce application

Steps:

1. Check product availability
2. Deduct money from the customer's account
3. Reduce stock quantity
4. Create order record

If payment fails, the order is not created.

All steps together form one transaction.

If we don't write these 4 SQL statements in a transaction, then if for some reason, SQL 4th fails (due to some internal server error) and SQL 2nd gets executed successfully, then money will be deducted from the customer's account, and the order will not be placed.

For writing a bunch of SQL statements in a transaction, below is the syntax:

```
BEGIN;

-- 1. Check product availability
SELECT stock
FROM products
WHERE product_id = 101
FOR UPDATE;

-- 2. Deduct money from customer account
UPDATE customers
SET balance = balance - 500
WHERE customer_id = 1
AND balance >= 500;

-- 3. Reduce stock quantity
UPDATE products
SET stock = stock - 1
WHERE product_id = 101
AND stock > 0;

-- 4. Create order record
INSERT INTO orders (customer_id, product_id, amount)
VALUES (1, 101, 500);

COMMIT;
```

To start a transaction, you need to specify BEGIN; and at the end write COMMIT; And all the SQL statements written between BEGIN and COMMIT will be a transaction.

As we discussed, all the 4 SQL statements will be executed in transaction, and either all of them will be successful or none of them.

Ticket Booking

When booking a train or flight ticket:

Steps:

1. Check seat availability
2. Block seat
3. Make payment
4. Confirm booking

If the payment fails, the seat is released. However, if these four SQL statements are not executed within a transaction and the third statement fails after the second one succeeds, you won't receive payment from the customer, and you'll end up losing one seat.

```
BEGIN;

-- 1. Check seat availability and lock it
SELECT id
FROM seats
WHERE trip_id = 1
      AND seat_number = 'A1'
      AND status = 'available'
FOR UPDATE;

-- 2. Block the seat
UPDATE seats
SET status = 'blocked'
WHERE trip_id = 1
      AND seat_number = 'A1';

-- 3. Create booking
INSERT INTO bookings (user_id, trip_id, seat_id, status)
VALUES (1, 1, 10, 'blocked');

-- 4. Make payment
INSERT INTO payments (booking_id, amount, status)
VALUES (1, 500.00, 'success');

-- 5. Confirm booking
UPDATE bookings
SET status = 'confirmed'
WHERE id = 1;

UPDATE seats
SET status = 'booked'
WHERE id = 10;

COMMIT;
```

Hope, the transaction's meaning is clear. Its a bunch of SQL statements that are executed as a single unit.

In PostgreSQL, even a single SQL statement is treated as a transaction.

Introduction to ACID Properties

ACID Properties are a bunch of guarantees that a database provides to ensure that transactions are processed reliably.

ACID properties guarantee that even if something goes wrong (like a power failure), the database remains in a consistent state.

The primary reason we use relational databases such as PostgreSQL, MySQL, Microsoft SQL Server, and Oracle Database is that they provide ACID guarantees.

The ACID properties are:

- **Atomicity (A)**
- **Consistency (C)**
- **Isolation (I)**
- **Durability (D)**

We will take a deep dive into each of these properties and get hands-on practice in PostgreSQL. You can choose any database you prefer; the concepts remain the same across platforms. Our examples will be demonstrated in the PostgreSQL terminal, but if you like, you can implement them in any programming language (such as Go, Java, TypeScript, etc.) using any ORM of your choice.

If you don't have PostgreSQL on your computer, then you can install it from [here](#).

Open psql and create a fresh DB:

```
CREATE DATABASE acid_tutorial;
```

We'll build a banking system

Go inside acid_tutorial DB by the following command:

```
\c acid_tutorial
```

Create an account table with columns named name and balance, indicating a person and their bank account balance.

```
CREATE TABLE accounts (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    balance INT NOT NULL  
);
```

```
INSERT INTO accounts (name, balance)
VALUES
('Shivam', 5000),
('Rohit', 2000),
('Aman', 3000),
('Rahul', 3000);
```

Now, we have an accounts table and some entries in it to play with.

```
acid_tutorial=# SELECT * FROM accounts;
[ id | name | balance
-----+-----+-----
  1 | Shivam |    5000
  2 | Rohit  |    2000
  3 | Aman   |    3000
  4 | Rahul  |    3000
(4 rows)
```

Run a transaction:

```
BEGIN;

UPDATE accounts SET balance = balance - 500 WHERE name='Rohit';
UPDATE accounts SET balance = balance + 500 WHERE name='Rahul';

COMMIT;
```

```
[acid_tutorial=# SELECT * FROM accounts;
 id | name | balance
-----+-----+-----
  1 | Shivam |    5000
  3 | Aman   |    3000
  2 | Rohit  |    1500
  4 | Rahul  |    3500
(4 rows)
```

Both SQL statements of the above applied successfully, i.e, Rohit (id = 2) balance deducted by 500, and Rahul (id=4) balance added by 500.

Atomicity

This is the same thing as we covered earlier.

“Either the entire transaction happens, or nothing happens.”

It means that if there are five SQL statements in a transaction, either all of them are executed successfully, or none of them take effect. For example, if the third SQL statement fails due to a network issue or power failure, all the previous statements in that transaction are rolled back. As a result, the system behaves as if the transaction never occurred.

I will try to simulate a failure condition with the following transaction:

```
BEGIN;  
  
UPDATE accounts SET balance = balance - 300 WHERE name='Shivam';  
  
-- simulate error  
SELECT 1/0;  
  
UPDATE accounts SET balance = balance + 300 WHERE name='Aman';  
  
COMMIT;
```

In the second line `SELECT 1/0;` gives error: ERROR: division by zero and the program crashes, so it will also rollback the first successfully executed SQL and the DB behave that this transaction never happened.

```

acid_tutorial=# SELECT * FROM accounts;
id | name  | balance
---+-----+-----
 1 | Shivam |    5000
 3 | Aman  |    3000
 2 | Rohit  |    1500
 4 | Rahul  |    3500
(4 rows)

acid_tutorial=# BEGIN;
BEGIN
acid_tutorial=##
acid_tutorial=## UPDATE accounts SET balance = balance - 300 WHERE name='Shivam';
UPDATE 1
acid_tutorial=##
acid_tutorial=## -- simulate error
acid_tutorial=## SELECT 1/0;
ERROR:  division by zero
acid_tutorial=!#
acid_tutorial=!# UPDATE accounts SET balance = balance + 300 WHERE name='Aman';
ERROR:  current transaction is aborted, commands ignored until end of transaction block
acid_tutorial=!#
acid_tutorial=!# COMMIT;
ROLLBACK
acid_tutorial=# SELECT * FROM accounts;
id | name  | balance
---+-----+-----
 1 | Shivam |    5000
 3 | Aman  |    3000
 2 | Rohit  |    1500
 4 | Rahul  |    3500
(4 rows)

acid_tutorial=# █

```

As you can see, the transaction was rolled back due to an error, so the balance that was supposed to be deducted from Shivam (id=1) was not updated. In this example, I manually simulated the error using `SELECT 1/0;`. In a real system, such errors could be caused by power cuts or other unexpected failures, which would result in the entire transaction being rolled back.

If we had written the two `UPDATE` statements without wrapping them in a transaction (i.e., without `BEGIN;` and `COMMIT;`), the first `UPDATE` would have executed while the second would not, leaving the database in an inconsistent state. This is precisely the situation we want to avoid.

Isolation

This determines how much one transaction can see or be affected by other transactions that are running at the same time.

The concept of isolation comes into play when two or more transactions run at the same time. In a real-world scenario, this could happen when multiple users send requests to a server

simultaneously, and the application server starts multiple transactions in the database at the same time.

A common question arises: If one transaction is still running and hasn't finished (i.e., its COMMIT statement hasn't been executed), can other transactions see its updates before it commits? As an application developer, you need to make this decision when designing your application. I'm going to explain all the possible scenarios that PostgreSQL provides. Once you understand these, you will have a clear idea of how to handle this in your application.

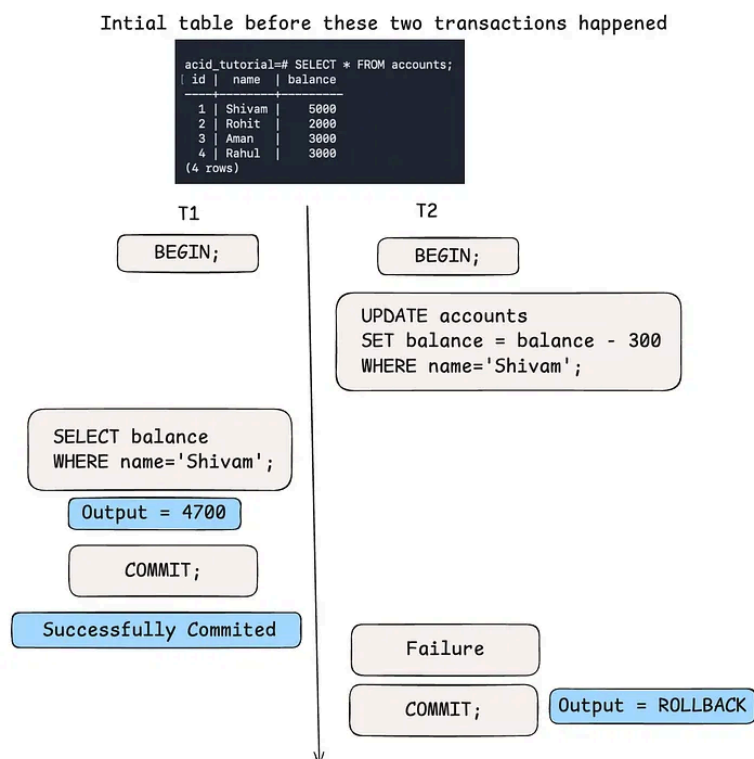
Three kinds of anomalies can happen when multiple transactions happen at the same time:

Dirty Read

A transaction (T1) reads the data that is being modified by an uncommitted concurrent transaction (T2). If the concurrent transaction (T2) eventually commits, the data read is valid. However, if it rolls back, the reading transaction (T1) would have accessed data that never actually existed, and this is called a dirty read.

The diagram below illustrates a dirty read scenario. The vertical axis represents time, which increases downward, while transactions T1 and T2 execute in parallel.

Press enter or click to view image in full size



As we can see in the image, T2 failed, and in the final state of DB, the balance of Shivam is still 5000, but T1 reads the value 4700 before T2 commits and returns it to the client, but the balance of Shivam is 5000 in the final DB. This is Dirty Read .

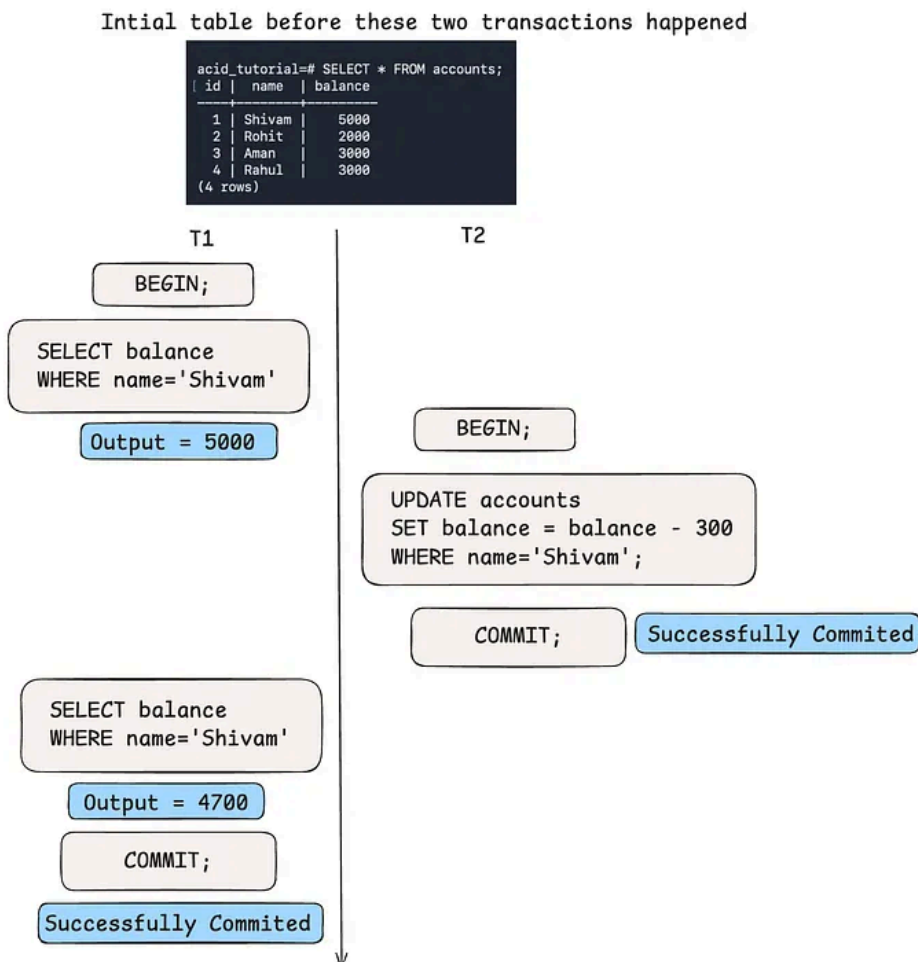
2. Nonrepeatable Read

It happens when a transaction reads the same piece of data twice, but gets different values each time because another transaction changed it in between.

For example:

- (i) Transaction A reads a value from the database.
- (ii) Transaction B updates that value and commits.
- (iii) Transaction A reads the same value again and sees the new value.

This can be confusing because it looks like the data “changed by itself” while Transaction A was running.



As we can see in the image, T1 hasn't done any update in the balance of Shivam still got different values because in between that time of two reads another transaction committed a successful UPDATE on that row. This is Nonrepeatable Read .

3. Phantom Read

It happens when a transaction runs a query twice and sees different rows each time, even though it didn't change anything itself. This usually happens because another transaction added or deleted rows in the meantime.

For example:

- (i) Transaction A runs a query to get all orders over \$100.
- (ii) Transaction B inserts a new order over \$100 and commits.
- (iii) Transaction A runs the same query again and now sees the new order like a "phantom" appeared.

Phantom reads are different from nonrepeatable reads because the number of rows changes, not just the values in existing rows.

Press enter or click to view image in full size

As we can see in the image, T1 hasn't performed any INSERT operations, yet it sees a new row when it queries again. This row wasn't present during T1's initial query. This phenomenon is known as a Phantom Read.

Isolation Levels

There are 4 levels of isolation in Databases that prevent the above anomalies:

- **READ UNCOMMITTED**
- **READ COMMITTED**
- **REPEATABLE READ**
- **SERIALIZABLE (strongest)**

PostgreSQL only provides 2nd, 3rd, and 4th isolation levels. 1st is not present in PostgreSQL as of now.

Below is the table that shows which isolation protects which anomaly:

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PG	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PG	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

Source: <https://www.postgresql.org/docs/current/transaction-iso.html>

Stricter isolation levels provide greater data consistency, but they can negatively impact performance. Therefore, the appropriate choice should be guided by the specific requirements and nature of the application.

Read Committed

This is the default transaction isolation level. If you do not specify an isolation level when starting a transaction, it will automatically begin with this level.

As we can see in the table, in this transaction we won't get Dirty Reads, but still we get Nonrepeatable Read and Phantom Read.

Syntax:

```
BEGIN;  
  
SELECT balance FROM accounts WHERE id = 1;  
UPDATE accounts SET balance = balance - 100 WHERE id=1;  
  
COMMIT;
```

To simulate this, we will open two simultaneous psql terminals and perform transactions:

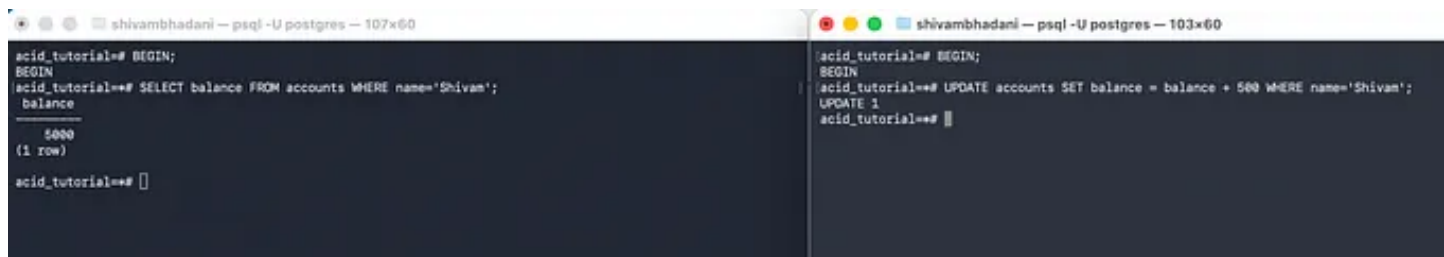
1. Started two transactions concurrently with BEGIN; keyword, and by default, both transactions started in Read Committed mode and read the balance of name=Shivam in left transaction.

Press enter or click to view image in full size



```
shivambhadani - psql -U postgres - 107x60  
acid_tutorial=# BEGIN;  
BEGIN  
acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';  
  balance  
-----  
    5000  
(1 row)  
acid_tutorial=#  
  
shivambhadani - psql -U postgres - 103x60  
acid_tutorial=# BEGIN;  
BEGIN  
acid_tutorial=#
```

2. Updated balance of Shivam by adding 500 to it in the right transaction.



```
shivambhadani - psql -U postgres - 107x60  
acid_tutorial=# BEGIN;  
BEGIN  
acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';  
  balance  
-----  
    5000  
(1 row)  
acid_tutorial=#  
  
shivambhadani - psql -U postgres - 103x60  
acid_tutorial=# BEGIN;  
BEGIN  
acid_tutorial=# UPDATE accounts SET balance = balance + 500 WHERE name='Shivam';  
UPDATE 1  
acid_tutorial=#
```

3. Read the balance of Shivam in the left transaction, and still got 5000 because we haven't COMMITTED the right transaction, and this proves that we are not getting Dirty Read



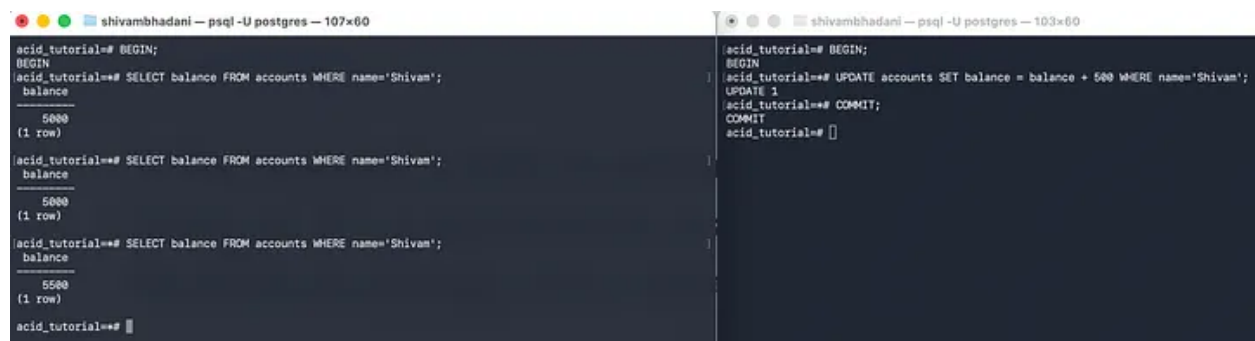
```
acid_tutorial=# BEGIN;
BEGIN
acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5000
(1 row)

acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5000
(1 row)

acid_tutorial=#
```

```
acid_tutorial=# BEGIN;
BEGIN
acid_tutorial=# UPDATE accounts SET balance = balance + 500 WHERE name='Shivam';
UPDATE 1
acid_tutorial=# COMMIT;
```

4. We committed the right transaction and read again the same row, i.e, the balance of Shivam and this time we are getting 5500 . This shows that we are getting the anomaly of non-repeatable read in this isolation level.



```
acid_tutorial=# BEGIN;
BEGIN
acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5000
(1 row)

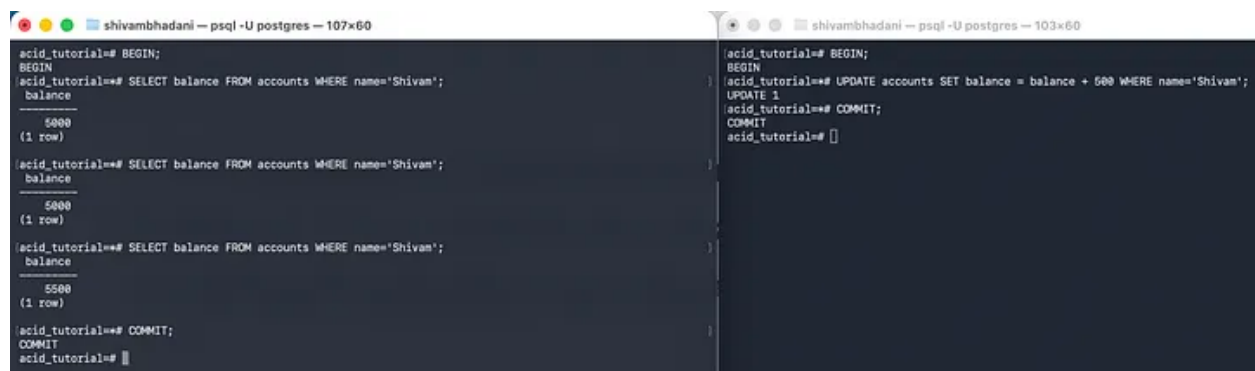
acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5000
(1 row)

acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5500
(1 row)

acid_tutorial=#
```

```
acid_tutorial=# BEGIN;
BEGIN
acid_tutorial=# UPDATE accounts SET balance = balance + 500 WHERE name='Shivam';
UPDATE 1
acid_tutorial=# COMMIT;
```

5. Finally, we also committed left transaction to make it complete.



```
acid_tutorial=# BEGIN;
BEGIN
acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5000
(1 row)

acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5000
(1 row)

acid_tutorial=# SELECT balance FROM accounts WHERE name='Shivam';
 balance
-----
 5500
(1 row)

acid_tutorial=# COMMIT;
```

```
acid_tutorial=# BEGIN;
BEGIN
acid_tutorial=# UPDATE accounts SET balance = balance + 500 WHERE name='Shivam';
UPDATE 1
acid_tutorial=# COMMIT;
```

As we can see, at this isolation level, we do not encounter dirty reads, but we do experience non-repeatable reads. If you simulate a phantom read, you will also observe that this anomaly occurs at this isolation level.

When to use Read Committed Isolation Level?

This is the most common and default isolation, so in 95% usecases this isolation level is sufficient.

Examples:

- E-commerce website product browsing
- User profile updates

And many more examples where this can be used.

Repeatable Read

It is a stricter isolation level than Read Committed that prevents dirty reads, nonrepeatable reads, and phantom reads.

Syntax:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
  
SELECT balance FROM accounts WHERE id = 1;  
UPDATE accounts SET balance = balance - 100 WHERE id=1;  
  
COMMIT;
```

Simulating this:

Terminal 1:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
SELECT balance FROM accounts WHERE name='Shivam';
```

Output = 5500

Terminal 2:

```
UPDATE accounts SET balance = balance + 500 WHERE name='Shivam';  
COMMIT;
```

Terminal 1:

```
SELECT balance FROM accounts WHERE name='Shivam';
```

Output = 5500

In reality, in the DB, currently, the balance of Shivam is changed to 6000

We can see that for both SELECT statements in terminal 1, we got the same value. This means it is protecting from Nonrepeatable Read. If you simulate a **phantom read**, you will also observe that it will also protect from phantom read.

When to use Repeatable Read Isolation Level?

Used when you need strong consistency.

Examples:

- Booking Systems: You check availability. Then insert the reservation.

Serializable

It is the strongest Isolation Level. It guarantees full serializability. Serializable means the database behaves as if transactions happened one at a time, preserving correctness even in concurrent execution.

Syntax:

```
BEGIN ISOLATION LEVEL SERIALIZABLE;

-- Your operations
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;
```

There are two types of Serializability:

- Conflict Serializable
- View Serializable

I won't go into the details of both Serializables, as it's not necessary for an application-layer developer to know.

If you set the Isolation Level to Serializable:

PostgreSQL might allow both transactions to start.

At commit, it checks for conflicts.

If a conflict is detected that would break serializability, one transaction is aborted with an error:

ERROR: could not serialize access due to concurrent update

Postgres checks for View Serializable.

```
-- T1
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT balance FROM accounts WHERE id = 1; -- reads 1000
UPDATE accounts SET balance = balance - 100 WHERE id = 1;

-- T2
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
UPDATE accounts SET balance = balance - 200 WHERE id = 1; -- writes same
```

```
row  
COMMIT;  
  
-- T1 tries to commit  
COMMIT; -- may fail with serialization error!
```

T1 read balance = 6000

T2 updated balance = 6000 -> 5800

When T1 tries to commit, PostgreSQL sees: "T1 read data that T2 modified concurrently, this could lead to a non-serializable result."

PostgreSQL aborts T1 to prevent a conflict.

DB behaves like transactions run one by one.

When to use Serializable Isolation Level?

Used in:

- Payments
- Inventory
- Financial ledgers

Where you need the strongest consistency.

Durability

"Once COMMIT happens, data will never be lost."

Postgres uses:

- WAL (Write Ahead Log)
- fsync
- disk flush

Flow:

- Transaction commits
- Changes written to WAL
- WAL flushed to disk
- Then commit returns

Even if the server crashes, WAL replays.

This was just an overview of Durability. We will explore its internals in much greater depth in the upcoming blogs.

In durability, just remember that if the COMMIT; statement is successfully executed, then no matter what, the data changes of that transaction will always persist.

Consistency

Consistency ensures that every transaction takes the database from one valid state to another valid state, following all rules, constraints, and schemas defined in the database.

Constraints include:

- Primary key and unique constraints
- Foreign key constraints
- Check constraints
- Data types
- Triggers and rules

Think of it like this: the database has rules for “valid states”, and consistency prevents transactions from violating these rules.

It's the job of the application developer who is creating the schema and writing the code to maintain the consistency.

Common methods to ensure consistency at the transaction level include the use of constraints and triggers.

a) Constraints

PostgreSQL enforces:

- **Primary Keys / Unique:** No duplicate entries
- **Foreign Keys:** No orphaned references
- **Check Constraints:** Custom validation rules

Example:

```
CREATE TABLE accounts (  
    id SERIAL PRIMARY KEY,  
    balance INT CHECK (balance >= 0)  
);  
  
INSERT INTO accounts (balance) VALUES (100); -- (executed successfully)  
INSERT INTO accounts (balance) VALUES (-50); -- (gives error) violates  
CHECK constraint
```

If a transaction violates a constraint, PostgreSQL aborts the transaction, rolling back automatically. This preserves consistency.

b) Triggers

Triggers allow custom logic to enforce complex business rules that constraints alone cannot capture.

```
CREATE FUNCTION ensure_balance() RETURNS trigger AS $$
BEGIN
    IF NEW.balance < 0 THEN
        RAISE EXCEPTION 'Balance cannot be negative';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_balance
BEFORE INSERT OR UPDATE ON accounts
FOR EACH ROW EXECUTE FUNCTION ensure_balance();
```

When performing INSERT, UPDATE operation:

```
INSERT INTO accounts (name, balance) VALUES ('Mohit', 100);
-- Works fine

INSERT INTO accounts (name, balance) VALUES ('Messi', -50);
-- ERROR: Balance cannot be negative
```

```
UPDATE accounts SET balance = -20 WHERE id = 1;
-- ERROR: Balance cannot be negative
```

Any transaction violating this rule is rolled back, keeping the database consistent.

To ensure consistency, a transaction must also adhere to Atomicity, Isolation, and Durability. If any of the four ACID properties fail, consistency breaks.

SQL INTERVIEW QUERIES

```
create table customers(
    customer_id int auto_increment primary key,
    name varchar(100),
    email varchar(50),
    phone_number varchar(20),
    city varchar(100)
```

```

);

create table tables(
    table_id int auto_increment primary key,
    name varchar(20),
    capacity int default 4,
    status ENUM('AVAILABLE', 'OCCUPIED', 'RESERVED'),
    created_at timestamp default current_timestamp,
    updated_at timestamp default current_timestamp on update
current_timestamp
);

create table orders(
    order_id int auto_increment primary key,
    order_type ENUM('DINE_IN', 'TAKE_AWAY', 'DELIVERY'),
    status ENUM('PLACED', 'PREPARING', 'READY', 'SERVED', 'CANCELLED',
'COMPLETED'),
    customer_id int,
    table_id int,
    total_amount decimal(10,2),
    tax_amount decimal(10,2),
    discount_amount decimal(10,2),
    final_amount decimal(10,2),
    payment_status ENUM('PENDING', 'FAILED', 'PAID'),
    payment_method ENUM('CASH', 'CARD', 'UPI'),
    created_at timestamp default current_timestamp,
    updated_at timestamp default current_timestamp on update
current_timestamp,
    foreign key (customer_id) references customers(customer_id),
    foreign key (table_id) references tables(table_id)
);

create table items(
    item_id int auto_increment primary key,
    name varchar(200),
    description varchar(200),
    veg_nonveg ENUM('VEG', 'NON VEG'),
    category ENUM('STARTERS', 'MAIN COURSE', 'DESSERTS', 'DRINKS', 'SOUPS'),
    status ENUM('AVAILABLE', 'NOT AVAILABLE'),
    price double(10,2),
    created_at timestamp default current_timestamp,
    updated_at timestamp default current_timestamp on update
current_timestamp

```

```
);
```

```
create table order_item(  
    id int auto_increment primary key,  
    order_id int,  
    item_id int,  
    quantity int,  
    total_price double(10,2),  
    foreign key (order_id) references orders(order_id),  
    foreign key (item_id) references items(item_id),  
    created_at timestamp default current_timestamp,  
    updated_at timestamp default current_timestamp on update  
current_timestamp  
);
```

```
INSERT INTO customers (name, email, phone_number, city) VALUES  
( 'Satya Surya', 'satya@gmail.com', '9876543210', 'Hyderabad'),  
( 'Rahul Sharma', 'rahul@gmail.com', '9123456780', 'Bangalore'),  
( 'Priya Reddy', 'priya@gmail.com', '9988776655', 'Chennai'),  
( 'Anil Kumar', 'anil@gmail.com', '9012345678', 'Mumbai'),  
( 'Sneha Patel', 'sneha@gmail.com', '9090909090', 'Pune');
```

```
INSERT INTO tables (name, capacity, status) VALUES  
( 'T1', 4, 'AVAILABLE'),  
( 'T2', 2, 'OCCUPIED'),  
( 'T3', 6, 'RESERVED'),  
( 'T4', 4, 'AVAILABLE'),  
( 'T5', 8, 'OCCUPIED');
```

```
INSERT INTO items (name, description, veg_nonveg, category, status, price)  
VALUES  
( 'Paneer Butter Masala', 'Creamy paneer curry', 'VEG', 'MAIN COURSE',  
'AVAILABLE', 250.00),  
( 'Chicken Biryani', 'Spicy chicken rice', 'NON VEG', 'MAIN COURSE',  
'AVAILABLE', 300.00),  
( 'Veg Manchurian', 'Fried veg balls', 'VEG', 'STARTERS', 'AVAILABLE',  
180.00),  
( 'Tomato Soup', 'Hot tomato soup', 'VEG', 'SOUPS', 'AVAILABLE', 120.00),  
( 'Ice Cream', 'Vanilla scoop', 'VEG', 'DESSERTS', 'AVAILABLE', 90.00),  
( 'Coke', 'Cold drink', 'VEG', 'DRINKS', 'AVAILABLE', 50.00);
```

```

INSERT INTO orders
(order_type, status, customer_id, table_id, total_amount, tax_amount,
discount_amount, final_amount, payment_status, payment_method)
VALUES
('DINE_IN', 'PLACED', 1, 2, 430.00, 30.00, 20.00, 440.00, 'PENDING',
'UPI'),
('TAKE_AWAY', 'PREPARING', 2, NULL, 300.00, 18.00, 0.00, 318.00, 'PAID',
'CARD'),
('DELIVERY', 'READY', 3, NULL, 390.00, 20.00, 10.00, 400.00, 'PAID',
'UPI'),
('DINE_IN', 'SERVED', 4, 5, 180.00, 10.00, 0.00, 190.00, 'PAID', 'CASH'),
('DINE_IN', 'COMPLETED', 5, 1, 340.00, 20.00, 15.00, 345.00, 'PAID',
'CARD');

INSERT INTO order_item (order_id, item_id, quantity, total_price) VALUES
(1, 1, 1, 250.00),
(1, 3, 1, 180.00),

(2, 2, 1, 300.00),

(3, 2, 1, 300.00),
(3, 6, 2, 100.00),

(4, 3, 1, 180.00),

(5, 1, 1, 250.00),
(5, 5, 1, 90.00);

```

GROUP BY & AGGREGATIONS

1. Find total revenue per day for paid orders.

```

select date(created_at) as order_date, sum(final_amount) as revenue_per_day
from orders where payment_status = 'PAID' group by date(created_at);

```

2. Find total revenue per city.

```

select sum(o.final_amount) as revenue_per_city, cus.city from orders o

```

```
join customers cus on cus.customer_id = o.customer_id
where o.payment_status = 'PAID'
group by cus.city;
```

3. Get top 5 customers based on total spending.

```
select sum(o.final_amount) as spending, cus.name from orders o
join customers cus on o.customer_id = cus.customer_id
where o.payment_status = 'PAID'
group by o.customer_id, cus.name
order by spending desc limit 5;
```

4. Find the most ordered item by quantity.

```
select sum(oi.quantity) as total_quantity, item.name from items item
join order_item oi on item.item_id = oi.item_id
group by oi.item_id, item.name
order by total_quantity desc limit 1;
```

5. Get category-wise revenue.

```
select sum(oi.total_price) as revenue, item.category
from order_item oi join items item on oi.item_id = item.item_id
join orders o on o.order_id = oi.order_id
where o.payment_status = 'PAID'
group by item.category;
```

6. Find average order value per customer.

```
select avg(o.final_amount) as average_order_value, cus.name from orders o
join customers cus on cus.customer_id = o.customer_id
where o.payment_status = 'PAID'
group by o.customer_id, cus.name;
```

7. Get the number of orders per order type (DINE_IN, DELIVERY, TAKEAWAY).

```
select count(*) as count_per_type, order_type
from orders group by order_type;
```



FILTERING (WHERE vs HAVING)

8. Find customers who spent more than ₹2000.

```
select cus.name, sum(o.final_amount) as spent_amount
from customers cus join orders o on o.customer_id = cus.customer_id
where o.payment_status='PAID'
group by cus.customer_id, cus.name
having sum(o.final_amount) > 2000;
```

9. Get items ordered more than 10 times.

```
select item.name, SUM(oi.quantity) AS total_quantity
from order_item oi join items item on oi.item_id = item.item_id
group by item.item_id, item.name
having SUM(oi.quantity) > 10;
```

10. Find orders where discount is greater than 20% of total amount.

```
select o.order_id, o.discount_amount from orders o
where o.discount_amount > 0.2*o.total_amount;
```

11. Get customers who placed at least 3 orders.

```
select cus.name, count(*) as total_orders
from orders o join customers cus on cus.customer_id = o.customer_id
where o.payment_status = 'PAID'
group by cus.customer_id, cus.name
having count(*) > 3;
```

12. Find cities with more than 5 customers.

```
select city, count(*) as no_of_customer from
customers group by city having count(*) > 5;
```

SORTING & RANKING

13. Rank customers based on total spending.

```
select name, total_spent, rank() over (order by total_spent desc) as
rank_no
from (
    select cus.name, sum(o.total_amount) as total_spent from orders o
    join customers cus on cus.customer_id = o.customer_id
    where o.payment_status = 'PAID'
    group by cus.customer_id, cus.name
) r;
```

- 14. Get top 3 highest value orders.
 - 15. Find latest 10 orders.
 - 16. Get second highest order amount.
 - 17. Rank items within each category by sales.
-

JOINS (VERY IMPORTANT)

- 18. Get complete order details with customer and item names.
- 19. Find customers who never placed an order.
- 20. Find orders that do not have any items.
- 21. Get all customers and their total order count (including zero).
- 22. Find items that were never ordered.
- 23. Get table-wise total revenue for dine-in orders.

1. Get complete order details with customer and item names

```
SELECT
  o.order_id,
  c.name AS customer_name,
  i.name AS item_name,
  oi.quantity,
  oi.total_price,
  o.final_amount,
  o.status
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
LEFT JOIN order_item oi ON o.order_id = oi.order_id
LEFT JOIN items i ON oi.item_id = i.item_id;
```



Why

- **LEFT JOIN** ensures even orders without items are included
 - Chain: orders → customers → order_items → items
-



2. Find customers who never placed an order

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```



Why

- Classic **anti-join**
 - No matching order → NULL
-

✓ 3. Find orders that do not have any items

```
SELECT o.*
FROM orders o
LEFT JOIN order_item oi ON o.order_id = oi.order_id
WHERE oi.order_id IS NULL;
```

💡 Why

- Orders exist but no item rows
-

✓ 4. Get all customers and their total order count (including zero)

```
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

💡 Why

- **LEFT JOIN** keeps all customers
 - **COUNT()** ignores NULL → gives 0 automatically
-

✓ 5. Find items that were never ordered

```
SELECT i.*
FROM items i
LEFT JOIN order_item oi ON i.item_id = oi.item_id
```

```
WHERE oi.item_id IS NULL;
```

💡 Why

- Items not present in `order_item`
-

✅ 6. Get table-wise total revenue for dine-in orders

```
SELECT
  t.table_id,
  t.name AS table_name,
  SUM(o.final_amount) AS total_revenue
FROM tables t
JOIN orders o ON t.table_id = o.table_id
WHERE o.order_type = 'DINE_IN'
GROUP BY t.table_id, t.name;
```

💡 Why

- Only dine-in orders considered
 - `SUM(final_amount)` = revenue per table
-

🔥 Bonus (Better Approach for Large Data)

👉 Using `NOT EXISTS` (more efficient)

Customers without orders

```
SELECT *
FROM customers c
```

```
WHERE NOT EXISTS (  
    SELECT 1  
    FROM orders o  
    WHERE o.customer_id = c.customer_id  
);
```

Orders without items

```
SELECT *  
FROM orders o  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM order_item oi  
    WHERE oi.order_id = o.order_id  
);
```

TRICKY / ADVANCED

- 24. Find duplicate customers based on email.
 - 25. Get highest order per customer.
 - 26. Find customers whose single order value is greater than their average order value.
 - 27. Get orders where total of order_items does not match orders.final_amount.
 - 28. Find gaps in order IDs (missing IDs).
 - 29. Find customers who ordered all available items at least once.
 - 30. Get consecutive orders placed by same customer within 10 minutes.
-

WINDOW FUNCTIONS

- 31. Calculate running total of revenue by date.
- 32. Find moving average of last 3 orders per customer.
- 33. Rank orders per day based on amount.
- 34. Get top-selling item per category using window functions.

1. Running Total of Revenue by Date

```
SELECT
  DATE(o.created_at) AS order_date,
  SUM(o.final_amount) AS daily_revenue,
  SUM(SUM(o.final_amount)) OVER (ORDER BY DATE(o.created_at)) AS
running_total
FROM orders o
WHERE o.status = 'COMPLETED'
GROUP BY DATE(o.created_at)
ORDER BY order_date;
```

Why

- `SUM(...)` → daily revenue
- `SUM(SUM(...)) OVER` → cumulative sum
- Window runs **ordered by date**

 This is a **classic cumulative KPI query**

2. Moving Average of Last 3 Orders per Customer

```
SELECT
  o.customer_id,
  o.order_id,
  o.final_amount,
  AVG(o.final_amount) OVER (
    PARTITION BY o.customer_id
    ORDER BY o.created_at
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg_last_3_orders
FROM orders o;
```

Why

- `PARTITION BY customer_id` → per customer

- **ROWS BETWEEN 2 PRECEDING** → last 3 rows (current + 2 previous)

👉 Used in:

- Customer spending trends
 - Fraud detection patterns
-

3. Rank Orders per Day Based on Amount

```
SELECT
  DATE(o.created_at) AS order_date,
  o.order_id,
  o.final_amount,
  RANK() OVER (
    PARTITION BY DATE(o.created_at)
    ORDER BY o.final_amount DESC
  ) AS rank_per_day
FROM orders o;
```

💡 **Difference you must know (interview trap):**

Function	Behavior
RANK()	Skips ranks (1,1,3)
DENSE_RANK()	No gaps (1,1,2)

ROW_NUMBER Unique sequence
R()

👉 Expect follow-up: “Which one will you use and why?”

🔥 4. Top-Selling Item per Category

```
SELECT *
FROM (
    SELECT
        i.category,
        i.item_id,
        i.name,
        SUM(oi.quantity) AS total_sold,
        RANK() OVER (
            PARTITION BY i.category
            ORDER BY SUM(oi.quantity) DESC
        ) AS rank_in_category
    FROM order_item oi
    JOIN items i ON oi.item_id = i.item_id
    GROUP BY i.category, i.item_id, i.name
) ranked
WHERE rank_in_category = 1;
```

💡 Why

- First aggregate → total sold
- Then rank within category
- Outer query filters top item

👉 Real-world use:

- Menu optimization
- Inventory planning



STORED PROCEDURES

- 35. Write a procedure to fetch all orders for a given customer.
 - 36. Write a procedure to calculate total revenue between two dates.
 - 37. Write a procedure to place an order with multiple items.
 - 38. Write a procedure to cancel an order and update status.
-



FUNCTIONS

- 39. Create a function to calculate tax for a given amount.
 - 40. Create a function to return total spending of a customer.
 - 41. Create a function to classify orders as HIGH/MEDIUM/LOW value.
-



TRIGGERS

- 42. Create a trigger to update order total when a new item is added.
 - 43. Create a trigger to prevent inserting order_item if order is CANCELLED.
 - 44. Create a trigger to log deleted orders into audit table.
 - 45. Create a trigger to update table status when dine-in order is placed.
-



REAL-WORLD SCENARIOS

- 46. Design query to get daily, weekly, monthly revenue in one result.
- 47. Find peak hours based on order volume.
- 48. Get most profitable item (consider quantity * price).
- 49. Identify inactive customers (no orders in last 3 months).
- 50. Detect fraudulent orders (very high value compared to avg).